

Oltre la ricerca classica

Ricerca locale, ricerca online (cenni)

Claudio Gallicchio, 2025/26
claudio.gallicchio@unipi.it



Intelligenza artificiale

Un approccio moderno

Volume 1

Quarta edizione

Stuart Russell, Peter Norvig
a cura di Francesco Amigoni

 Pearson

MyLab Codice per accedere
alla piattaforma

Riferimenti

AIMA Cap. 4

Sez. 4.1, 4.2, 4.3

Risolutori «classici»

Gli agenti *risolutori di problemi* “classici” assumono:

- Ambienti **completamente osservabili**
- Ambienti **deterministici**
- Di essere nelle condizioni di produrre *offline* un piano (una sequenza di azioni) che può essere eseguito senza imprevisti per raggiungere l'obiettivo.

Verso ambienti più realistici

Finora: problemi in ambienti completamente osservabili, deterministici, statici, noti, in cui la soluzione è una sequenza di azioni.

- cerchiamo un **buono stato** senza preoccuparci del cammino per raggiungerlo
- spazi **discreti** e spazi **continui**

Ricerca locale

Verso ambienti più realistici

Rilassare il vincolo di determinismo.

- In un mondo non deterministico, l'agente ha bisogno di un **piano condizionale**

- eseguire azioni diverse in base a quello che osserva
 - ✓ piani condizionali, ricerca AND-OR, stati credenza

Ricerca con azioni non
deterministiche



Verso ambienti più realistici

Rilassare il vincolo di osservabilità completa.

- In caso di osservabilità parziale l'agente deve tenere traccia degli stati in cui **potrebbe** trovarsi

Ricerca con osservazioni parziali (AIMA, Sez 4.4)

Rilassare il vincolo di essere in un ambiente noto.

- Guidare l'agente attraverso uno **spazio sconosciuto**, mediante ricerca **online**.

Ricerca online e ambienti sconosciuti (AIMA, Sez 4.5)

Ricerca locale e problemi di ottimizzazione

Gli algoritmi visti esplorano gli spazi di ricerca alla ricerca di un goal e **restituiscono un cammino soluzione.**

- es., andare da Arad a Bucarest

Gli algoritmi visti esplorano gli spazi di ricerca alla ricerca di un goal e restituiscono un cammino soluzione.

- es., andare da Arad a Bucarest

A volte siamo **interessati solo allo stato finale**, non al cammino percorso per raggiungerlo.

- es., problema delle 8 regine

Ricerca locale

Gli **algoritmi di ricerca locale** operano a partire da uno stato iniziale e procedono verso gli stati adiacenti

- senza tenere traccia dei cammini
- senza tenere traccia degli stati già raggiunti

Ricerca locale

Non sono sistematici



- potrebbero non esplorare mai una porzione dello spazio degli stati dove risiede una soluzione



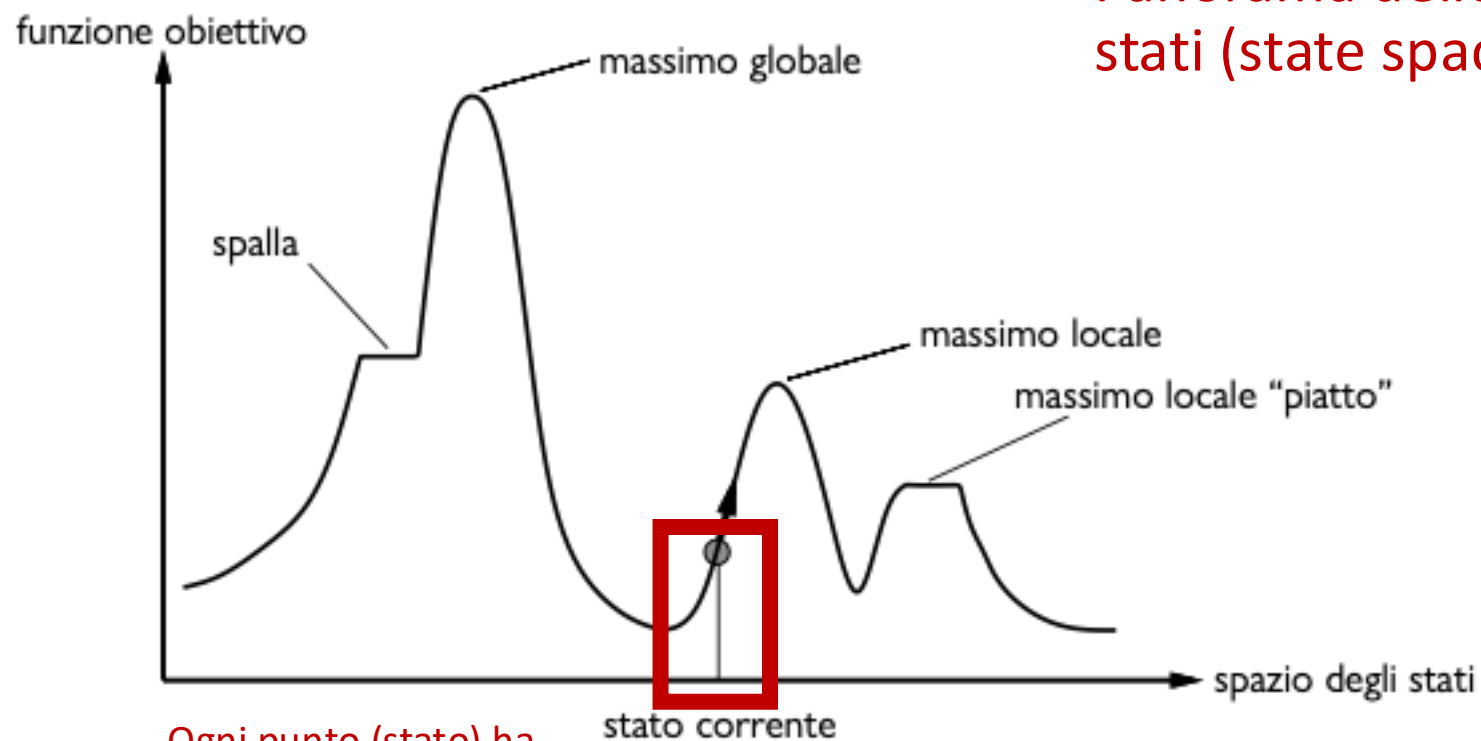
1. Usano **pochissima memoria**
2. Riescono a trovare soluzioni ragionevoli in **spazi degli stati** così **grandi** (anche infiniti) per cui non sarebbe praticabile usare algoritmi sistematici

Problemi di ottimizzazione

Lo scopo è trovare lo stato migliore secondo una
funzione obiettivo.

Problemi di ottimizzazione

Lo scopo è trovare lo stato migliore secondo una **funzione obiettivo**.



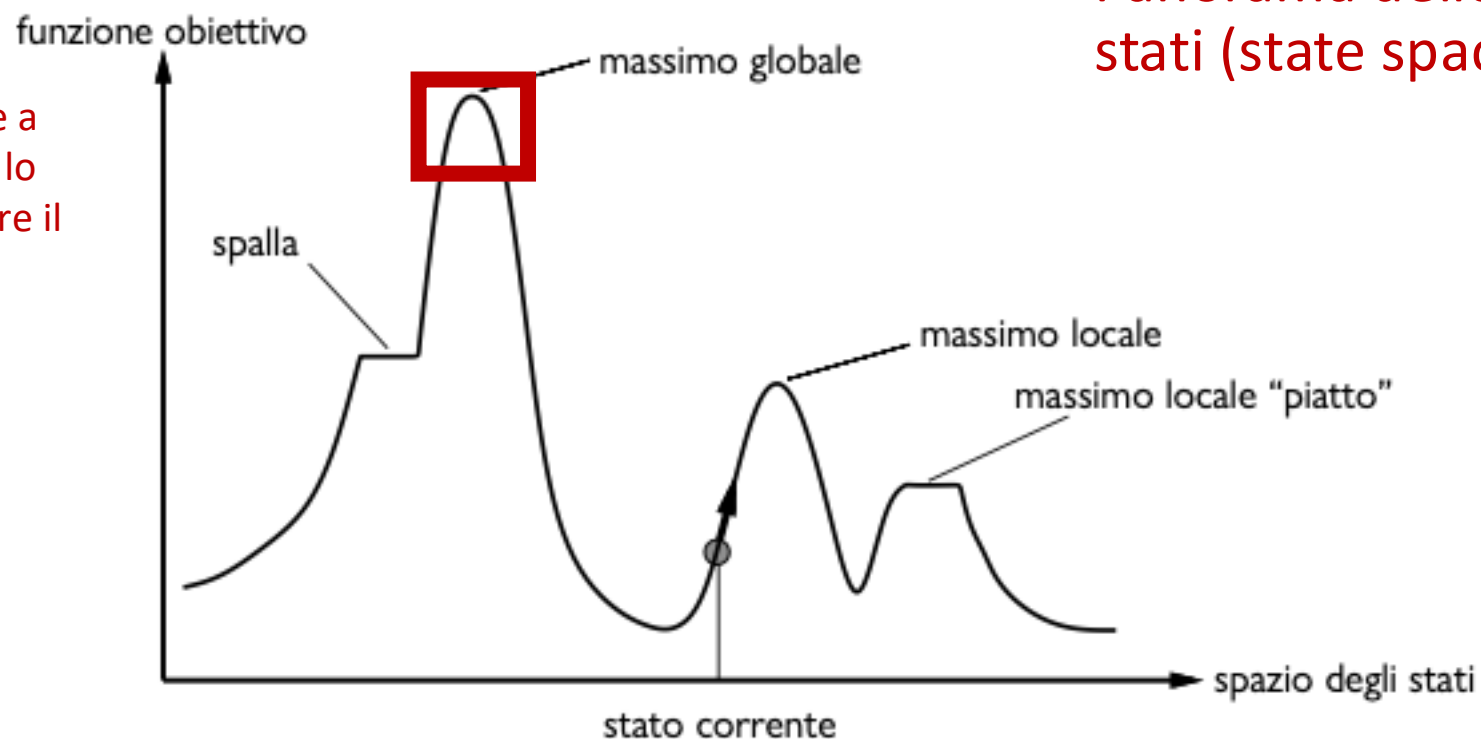
Ogni punto (stato) ha
un'altezza in base alla
funzione obiettivo

Problemi di ottimizzazione

Lo scopo è trovare lo stato migliore secondo una funzione obiettivo.

Se l'altezza corrisponde a una funzione obiettivo lo scopo è quello di trovare il picco più alto.

Hill climbing.



Panorama dello spazio degli stati (state space landscape)

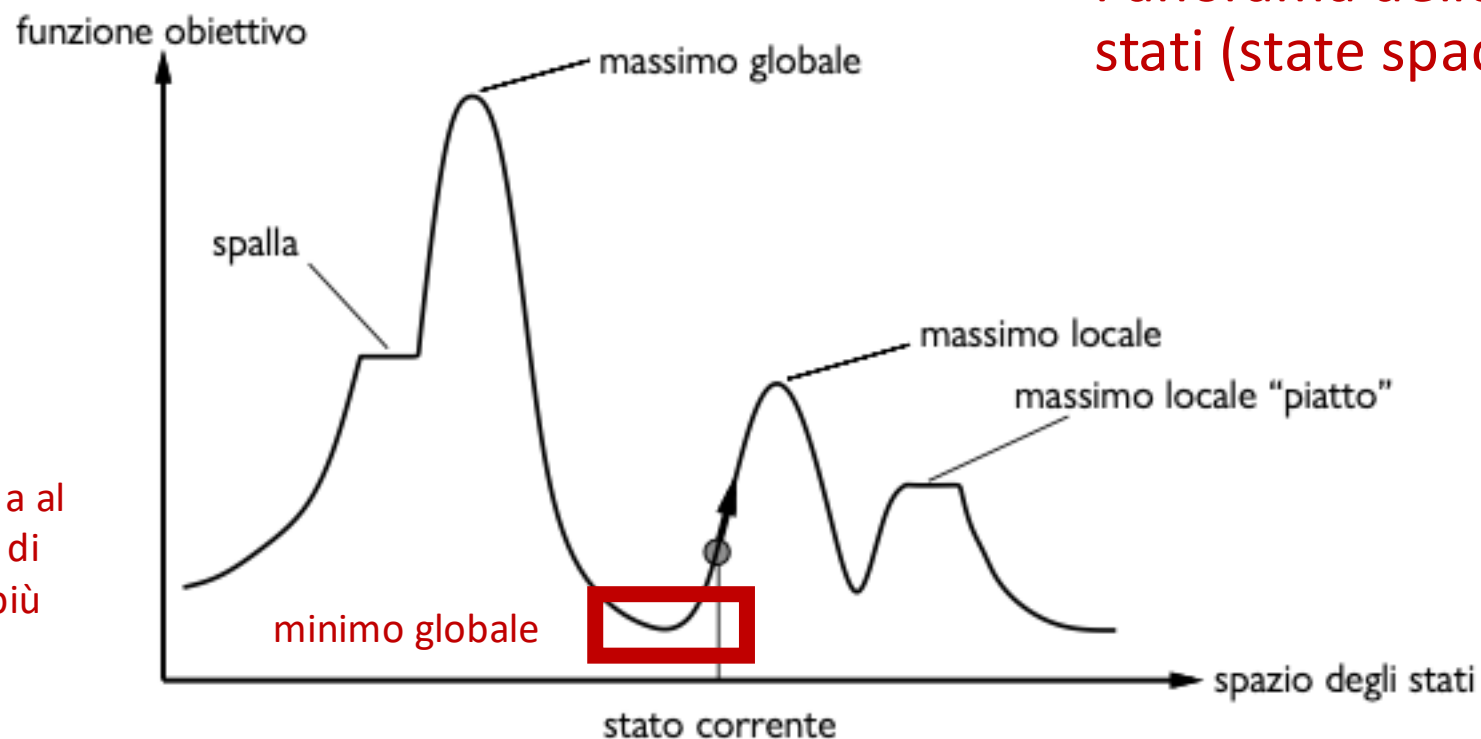
Problemi di ottimizzazione

Lo scopo è trovare lo stato migliore secondo una funzione obiettivo.

Panorama dello spazio degli stati (state space landscape)

Se l'altezza corrisponde a al costo, lo scopo è quello di trovare l'avvallamento più profondo.

Gradient descent.



Ricerca hill climbing



Hill climbing

Ricerca locale **greedy**.

- L'algoritmo tiene **traccia** solo dello **stato corrente**
- A ogni iterazione **passa** allo stato vicino con valore più **alto**
 - Punta nella direzione dell'**ascesa più ripida** (steepest ascent) – hill climbing a salita rapida / ripida
 - Alternative
 - ✓ Uno a caso tra quelli che salgono: hill climbing stocastico
 - ✓ Il primo generato: hill climbing con prima scelta

Hill climbing

Trovare il massimo locale.

function HILL-CLIMBING(*problema*) **returns** uno stato che è un massimo locale

corrente $\leftarrow$ *problema*.STATOINIZIALE

while true do

vicino $\leftarrow$ lo stato successore di *corrente* di valore più alto

if VALORE(*vicino*) $\leq$ VALORE(*corrente*) **then return** *corrente*

corrente $\leftarrow$ *vicino*

Se tutti i vicini hanno un valore più basso
allora l'algoritmo è arrivato ad un massimo
locale.

Hill climbing

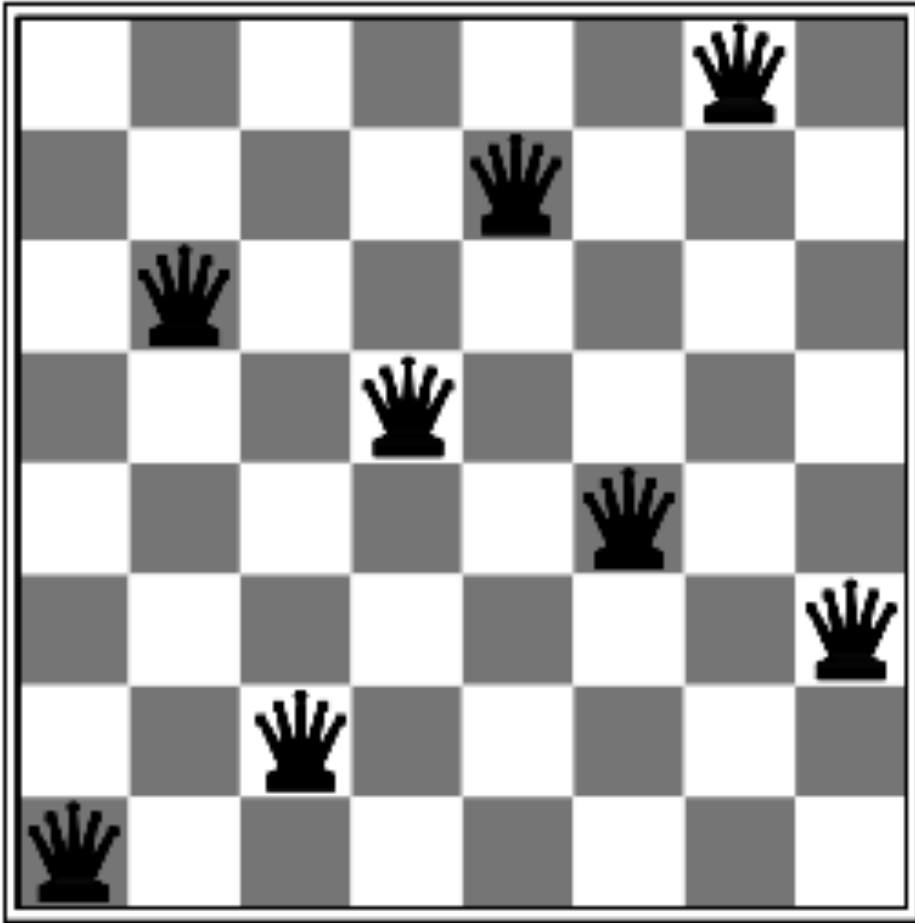
```
function HILL-CLIMBING(problema) returns uno stato che è un massimo locale  
  corrente  $\leftarrow$  problema.STATOINIZIALE  
  while true do  
    vicino  $\leftarrow$  lo stato successore di corrente di valore più alto  
    if VALORE(vicino)  $\leq$  VALORE(corrente) then return corrente  
    corrente  $\leftarrow$  vicino
```

Se invece volessimo trovare un minimo locale?

Usiamo come funzione obiettivo il valore negativo di una funzione di costo euristica.

- In questo modo si sale localmente allo stato con la minima distanza euristica dall'obiettivo.

8 regine



Formulazione a stato completo = ogni stato ha tutti i componenti di una soluzione.

- Ogni stato ha 8 regine sulla scacchiera, una per colonna
- Stato iniziale scelto a caso
- Successori: stati ottenibili muovendo una sola regina, in un'altra casella della colonna in cui si trova

Funzione costo euristico h = numero di coppie di regine che si attaccano

8 regine

Funzione costo euristico h = numero di coppie di regine che si attaccano

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14		13	16	13	16
	14	17	15		14	16	16
17		16	18	15		15	
18	14		15	15	14		16
14	14	13	17	12	14	12	18

Valore di h mostrato per ogni possibile successore ottenuto muovendo una regina nella sua colonna.

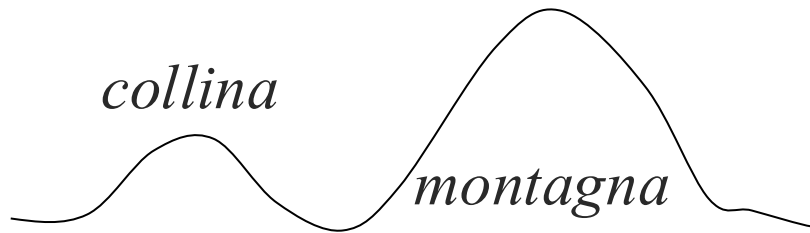
In soli 5 passi si può raggiungere uno stato in cui $h = 1$.

In questo stato, $h = 17$

Hill climbing: problemi

Se la funzione è da ottimizzare i picchi sono massimi locali o soluzioni ottimali

Massimo locale



Ogni azione nello stato corrente
può solo peggiorare la situazione

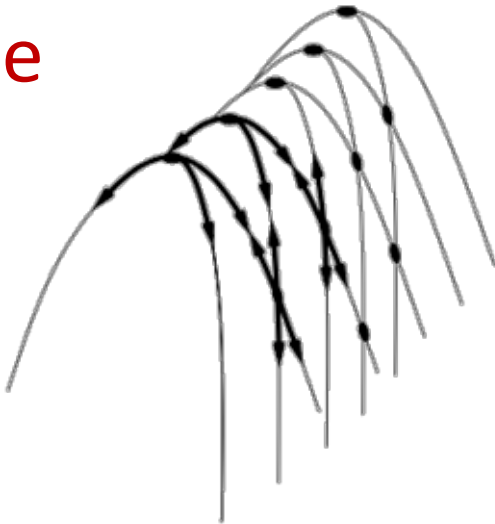
Un picco più alto degli stati vicini, ma inferiore al massimo globale.

Hill climbing nelle vicinanze di un massimo locale viene attratto verso il picco, restando bloccato senza poter individuare il massimo globale.

Hill climbing: problemi

Se la funzione è da ottimizzare i picchi sono massimi locali o soluzioni ottimali

Creste



una regione in cui la direzione del miglioramento non è direttamente accessibile con piccoli passi

Sequenza di massimi locali molto difficili da esplorare.

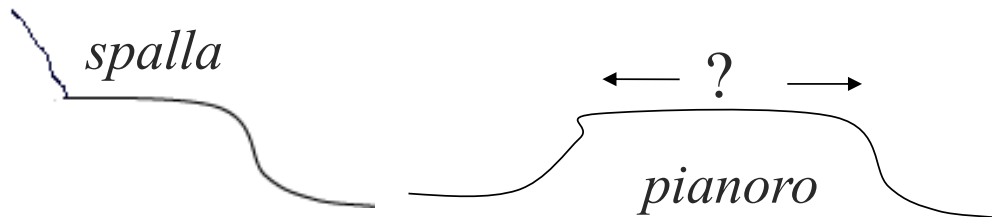
Difficili da esplorare: scendere e poi risalire

Da ogni massimo locale ogni azione punta “verso valle”.

Hill climbing: problemi

Se la funzione è da ottimizzare i picchi sono massimi locali o soluzioni ottimali

Plateau

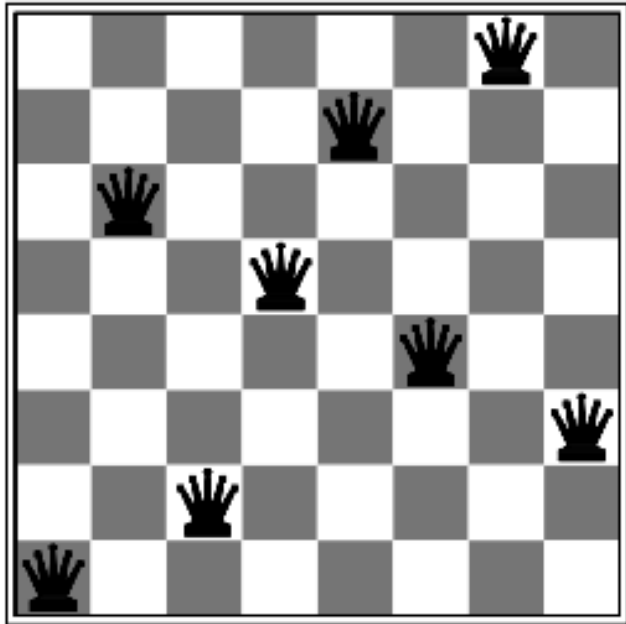


Area piatta del panorama dello spazio degli stati

- Massimo locale piatto da cui non è possibile fare ulteriori progressi
- Spalla (da cui potrebbe essere possibile risalire)

Hill climbing: problemi

L'algoritmo raggiunge un punto dal quale non riesce a compiere ulteriori progressi.



Partendo da uno stato generato casualmente:

- si blocca nell'86% dei casi
- risolve solo il 14% delle istanze
- molto veloce: in media solo 4 passi per trovare una soluzione
 - 3 passi quando si blocca
 - spazio degli stati enorme: ≈ 17 milioni di stati

Miglioramenti: mossa laterale

Idea: consentire (un numero limitato di) «mosse laterali» per affrontare i plateau.

- L'algoritmo può permettere alcuni passi in cui la funzione obiettivo **rimane invariata**.
 - Questo consente all'algoritmo di **esplorare l'area circostante** nella speranza di trovare un percorso che lo porti fuori dal plateau e verso un miglioramento.
- Porre un limite al numero di mosse laterali consecutive (es. 100)
 - 8 regine: la percentuale di istanze di problemi risolte sale dal 14% al 94%
 - Media 21 passi per ogni successo e 64 per ogni fallimento

Miglioramenti: hill climbing stocastico

Idea: scegliere a caso tra tutte le mosse che vanno nella direzione della funzione crescente (che puntano verso l'alto)

- La probabilità di scegliere ogni mossa può essere influenzata dalla pendenza delle mosse
 - probabilità di scelta maggiore per mosse che portano più in alto
- Solitamente: converge più lentamente, ma può trovare soluzioni migliori

Miglioramenti: hill climbing con prima scelta

Idea: generare casualmente i successori dello stato attuale fino a ottenerne uno preferibile allo stato corrente.

- Utile ad es., quando ogni stato può avere molti successori
 - migliaia e oltre

Miglioramenti: hill climbing con riavvio casuale

Idea: condurre una serie di ricerche hill climbing partendo da stati iniziali generati casualmente, fino a che non viene raggiunto un obiettivo.

- Completo con probabilità 1
 - prima o poi genera lo stato obiettivo come stato iniziale
- Numero atteso di riavvii = $1/p$, dove p = probabilità di successo per ogni ricerca hill climbing
 - es., in 8 regine senza mosse laterali $p \approx 0.14$ quindi ci vorranno 7 iterazioni (6 fallimenti e un successo)
 - anche con 3 milioni di regine, soluzione in pochi secondi

Successo dipende dalla forma del panorama dello spazio degli stati.

- se ci sono pochi plateau e massimi locali: hill climbing con riavvio casuale trova soluzione velocemente
- problemi reali hanno solitamente moltissimi massimi locali
 - spesso è comunque possibile trovare un massimo locale ragionevolmente buono con un numero piccolo di riavvii.

Simulated annealing

Tempra simulata

Idea

- **Hill climbing**: non scendere mai a valle
 - rischia di rimanere bloccato in un massimo locale
- **Esplorazione casuale**: scegliere un successore a caso
 - prima o poi trova il massimo globale, ma inefficiente

Idea: combinare in qualche modo hill climbing ed esplorazione casuale

Simulated annealing (tempra)



Tempra (metallurgia)

Processo usato per indurire i metalli (o il vetro) riscaldandoli ad altissime temperature e raffreddandoli gradualmente, consentendo al materiale di cristallizzare in uno stato di bassa energia.

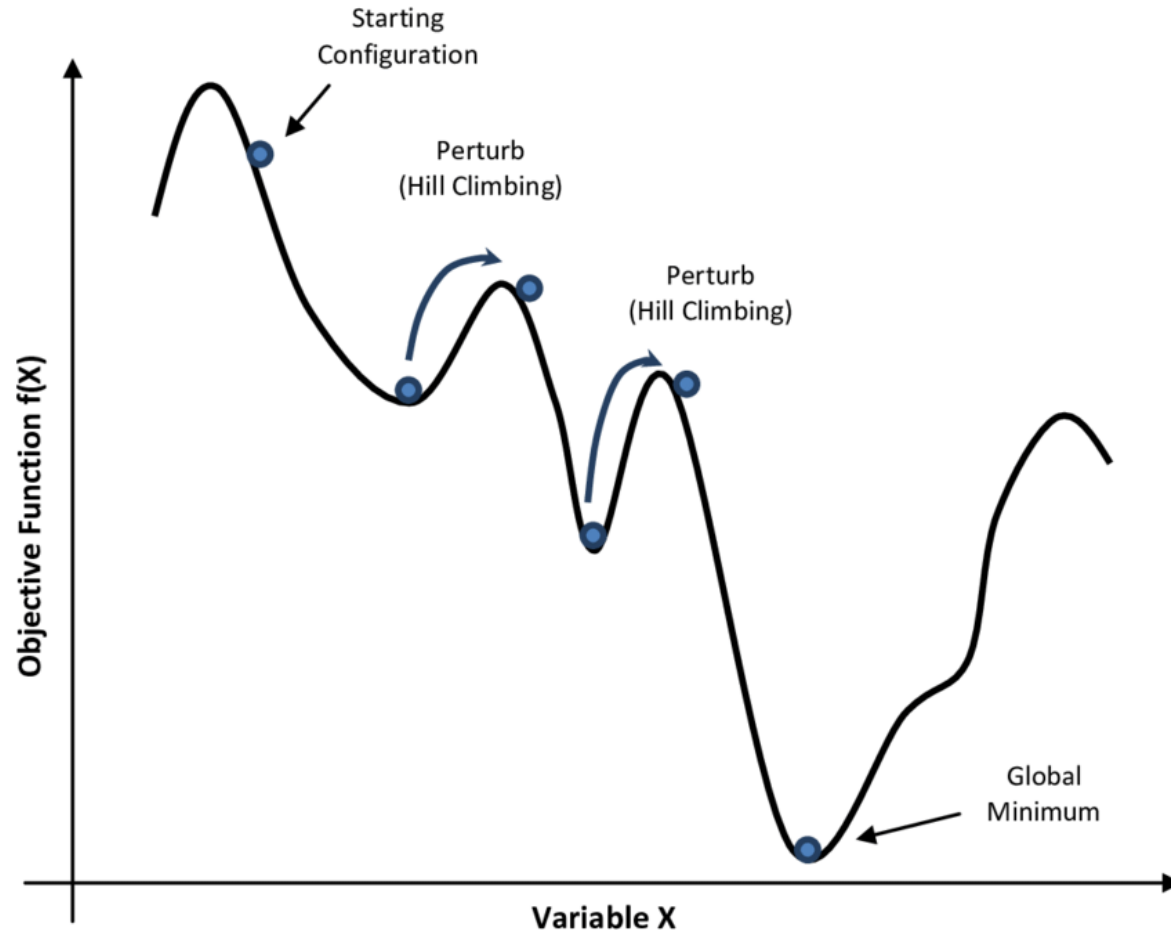
Optimization by Simulated Annealing

S. Kirkpatrick, C. D. Gelatt, Jr., M. P. Vecchi

with N , so that in practice exact solutions can be attempted only on problems involving a few hundred cities or less. The traveling salesman belongs to the large class of NP-complete (nondeterministic polynomial time complete) problems, which has received extensive study in the past 10 years (3). No method for exact solution with a computing effort bounded by a power of N has been found for any of these problems, but if such a solution were found, it could be mapped into a procedure for solving all members of the class. It is not known

Kirkpatrick, Scott, C. Daniel Gelatt Jr, and Mario P. Vecchi. "Optimization by simulated annealing." science 220.4598 (1983): 671-680.

Summary. There is a deep and useful connection between statistical mechanics (the behavior of systems with many degrees of freedom in thermal equilibrium at a finite temperature) and multivariate or combinatorial optimization (finding the minimum of a given function depending on many parameters). A detailed analogy with annealing in solids provides a framework for optimization of the properties of very large and complex systems. This connection to statistical mechanics exposes new information and provides an unfamiliar perspective on traditional optimization problems and methods.



Punto di vista: discesa del
gradiente
(minimizzazione del
costo)

Idea: scuotere lo stato
per farlo uscire da un
minimo locale.

Simulated annealing

- Struttura simile a quella dell'hill climbing
- Invece che la mossa migliore, viene scelta una mossa casuale
 - se la mossa migliora la situazione viene accettata
 - altrimenti, viene accettata con probabilità < 1

Simulated annealing

La probabilità di scegliere una mossa che peggiora la situazione

- **Decresce** esponenzialmente **con la «cattiva qualità» della mossa ΔE** (di quanto peggiora la valutazione?)
- **Decresce con la «temperatura» T** , che scende costantemente
 - le mosse «cattive» sono accettate più facilmente all'inizio (con T alta)
 - progressivamente con minor probabilità (per valori di T che diminuiscono)

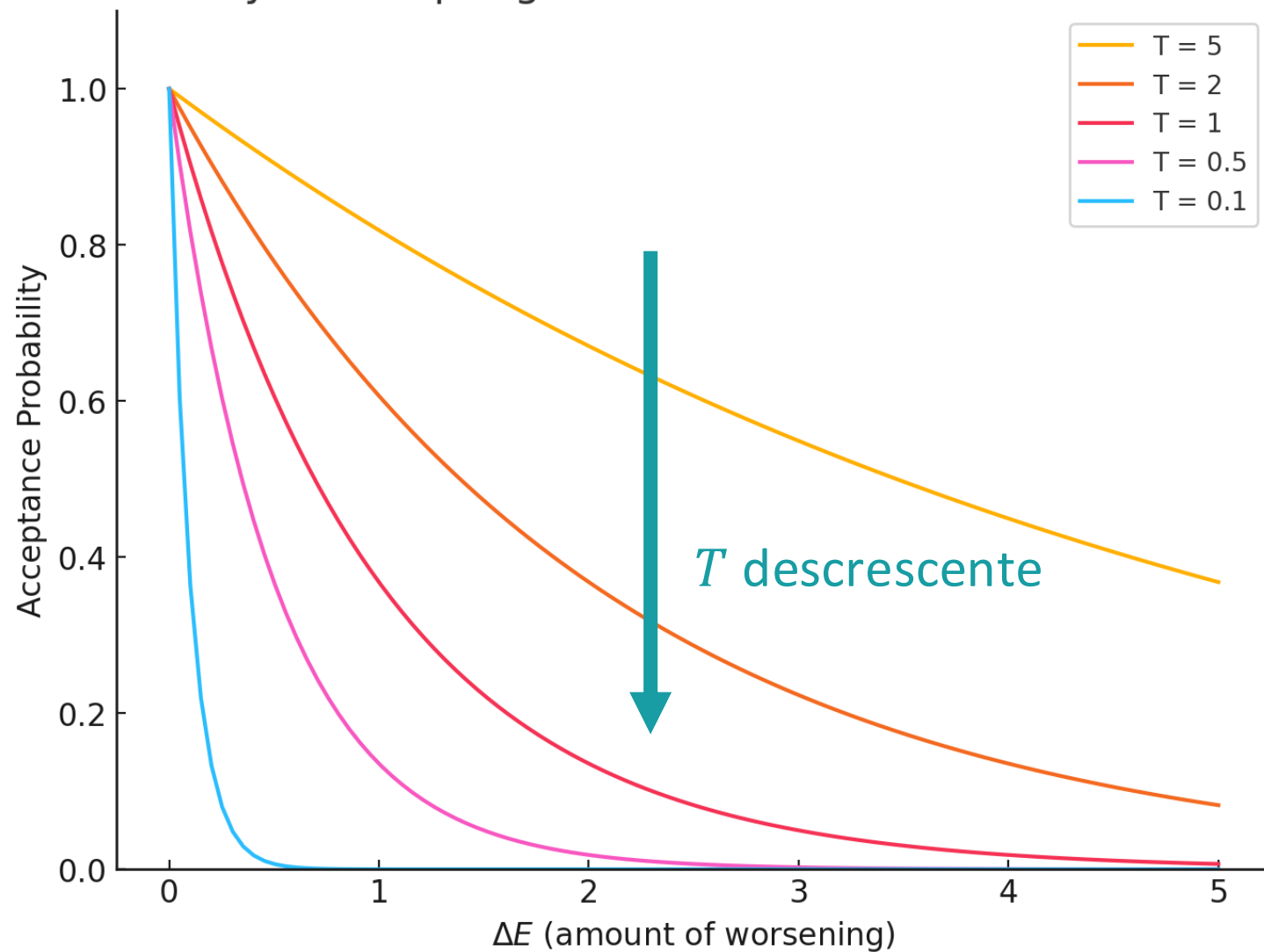
Simulated annealing

La probabilità di scegliere una mossa che peggiora la situazione

- Decresce esponenzialmente con la «cattiva qualità» della mossa ΔE (di quanto peggiora la valutazione?)
- Descresce con la «temperatura» T , che scende costantemente

$$prob = e^{-\frac{\Delta E}{T}}$$

Probability of Accepting a Worse Move in Simulated Annealing



$$prob = e^{-\frac{\Delta E}{T}}$$

Simulated annealing

La temperatura T diminuisce con una certa **velocità di raffreddamento**.

- Se T viene fatta decrescere abbastanza lentamente l'algoritmo troverà un massimo globale con probabilità che tende a 1

Simulated annealing (minimizzazione)

```
function simulated-annealing(problema, velocità_raffreddamento) returns uno stato soluzione
  corrente  $\leftarrow$  problema.StatoIniziale
  for t  $\leftarrow$  1 to  $\infty$  do
    T  $\leftarrow$  velocità_raffreddamento[t]
    if T = 0 then return corrente
    successivo  $\leftarrow$  un successore scelto a caso di corrente
     $\Delta E \leftarrow \text{Valore}(\text{successivo}) - \text{Valore}(\text{corrente})$ 
    if  $\Delta E < 0$  then corrente  $\leftarrow$  successivo
    else corrente  $\leftarrow$  successivo solo con probabilità  $\exp(-\Delta E/T)$ 
```

Di quanto peggioro

Simulated annealing (minimizzazione)

```
function simulated-annealing(problema, velocità_raffreddamento) returns uno stato soluzione
  corrente  $\leftarrow$  problema.StatoIniziale
  for t  $\leftarrow$  1 to  $\infty$  do
    T  $\leftarrow$  velocità_raffreddamento[t]
    if T = 0 then return corrente
    successivo  $\leftarrow$  un successore scelto a caso di corrente
     $\Delta E \leftarrow$  Valore(successivo) - Valore(corrente)
    if  $\Delta E < 0$  then corrente  $\leftarrow$  successivo
    else corrente  $\leftarrow$  successivo solo con probabilità  $\exp(-\Delta E/T)$ 
```

In questo caso migliore e quindi vado
nello stato successivo certamente

Simulated annealing (minimizzazione)

```
function simulated-annealing(problema, velocità_raffreddamento) returns uno stato soluzione  
  corrente  $\leftarrow$  problema.StatoIniziale  
  for t  $\leftarrow$  1 to  $\infty$  do  
    T  $\leftarrow$  velocità_raffreddamento[t]  
    if T = 0 then return corrente  
    successivo  $\leftarrow$  un successore scelto a caso di corrente  
     $\Delta E \leftarrow$  Valore(successivo) - Valore(corrente)  
    if  $\Delta E < 0$  then corrente  $\leftarrow$  successivo  
    else corrente  $\leftarrow$  successivo solo con probabilità  $\exp(-\Delta E/T)$ 
```

In questo caso peggioro e quindi vado
nello stato successivo con una probabilità
 $\exp(-\Delta E/T)$

Simulated annealing (minimizzazione)

```
function simulated-annealing(problema, velocità_raffreddamento) returns uno stato soluzione  
  corrente  $\leftarrow$  problema.StatoIniziale  
  for t  $\leftarrow$  1 to  $\infty$  do  
    T  $\leftarrow$  velocità_raffreddamento[t]
```

Esercizio: versione per massimizzazione

```
   $\Delta E \leftarrow$  Valore(successivo) - Valore(corrente)  
  if  $\Delta E < 0$  then corrente  $\leftarrow$  successivo  
  else corrente  $\leftarrow$  successivo solo con probabilità  $\exp(-\Delta E/T)$ 
```

Tempra simulata: parametri

- Valore iniziale e decremento di T sono iper-parametri
- Valori per T determinati sperimentalmente:
 - il valore iniziale di T è tale che per valori medi di ΔE , $p = e^{\Delta E/T}$ sia all'incirca 0.5



Ricerca local beam

Ricerca local beam

Versione locale della beam search

Idea: nella ricerca locale, tenere traccia di k stati anziché solo di 1.

- Si inizia con k stati generati casualmente
- Ad ogni passo, si generano i successori di tutti i k stati
 - se uno qualsiasi è obiettivo, l'algoritmo lo restituisce e termina
 - altrimenti, sceglie i k successori migliori dalla lista di tutti i successori e ricomincia

Ricerca local beam

- Diverso da k ricerche a riavvio casuale in parallelo
 - viene passata informazione utile da un thread all'altro
- Abbandona rapidamente ricerche infruttuose e sposta le sue risorse dove si stanno verificando i progressi maggiori

Ricerca local beam stocastica

- Local beam può soffrire di **carenza di diversificazione** tra i k stati, che possono concentrarsi in una piccola regione dello spazio degli stati
- **Variante stocastica**: invece di scegliere i migliori k successori, **si scelgono i successori con probabilità proporzionale al loro valore**

Algoritmi evolutivi

Varianti della ricerca beam stocastica, motivati dalla **metafora della selezione naturale in biologia**.

- una popolazione di individui (**stati**)
- gli individui più adatti (**valore più alto**)
- producono prole (**stati successori**) che formano la prossima generazione
 - **ricombinazione**: processo in cui gli individui più adatti di una generazione si combinano nei nuovi individui della generazione successiva

Varianti della ricerca beam stocastica, motivati dalla metafora della selezione naturale in biologia.

- una popolazione di individui (stati)
- gli individui più adatti (valore più alto)
- producono prole (stati successivi) che formano la prossima generazione
 - **ricombinazione**: processo in cui gli individui più adatti di una generazione si combinano nei nuovi individui della generazione successiva

Gli algoritmi evolutivi si differenziano tra loro per diversi aspetti

Caratteristiche

- **Dimensione** della popolazione
- **Rappresentazione** di ciascun individuo.
 - ogni individuo è una stringa su un alfabeto finito (come il DNA)
 - ✓ strategie evolutive: un individuo è una sequenza di numeri
 - ✓ programmazione genetica: un individuo è un programma per computer

Caratteristiche

- **Numero di genitori** che si uniscono per generare la prole (ρ)
 - $\rho = 2$ due «genitori» combinano i loro «geni» (parti della loro rappresentazione) per generare la prole
- Processo di **selezione** per scegliere gli individui che saranno genitori della generazione successiva
 - selezionarli tra gli individui con probabilità proporzionale al loro punteggio di adattamento (**fitness**)

Caratteristiche

- Procedura di **ricombinazione**
 - (con $\rho = 2$) selezionare un **punto di crossover** per suddividere ognuna delle stringhe genitori e poi ricombinare le parti per formare due figli
 - ✓ figlio 1: prima parte del genitore 1 e seconda parte del genitore 2
 - ✓ figlio 2: prima parte del genitore 2 e seconda parte del genitore 1

Caratteristiche

- **Tasso di mutazione:** determina la frequenza con cui la prole presenta mutazioni (genetiche) casuali della loro rappresentazione
 - una volta generata la prole, ogni bit che la compone viene invertito con probabilità = tasso di mutazione

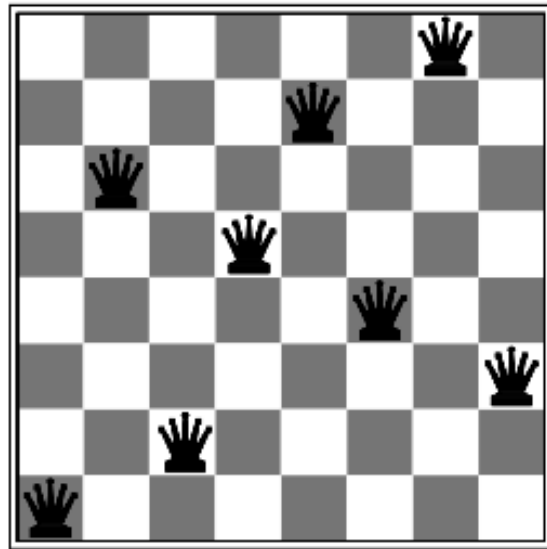
Caratteristiche

- Formazione della **nuova generazione**.
 - Può essere semplicemente la nuova prole
 - Può contenere alcuni genitori con il migliore punteggio dalla generazione precedente (**elitismo**)
 - ✓ garantisce che l'adattabilità totale non diminuisca
 - Si può eliminare ogni individuo al di sotto di una certa soglia di fitness (**abbattimento**)

Esempio 8 regine

Rappresentazione: stringhe di 8 cifre

- cifra i -esima: riga in cui si trova la regina sulla colonna i -esima

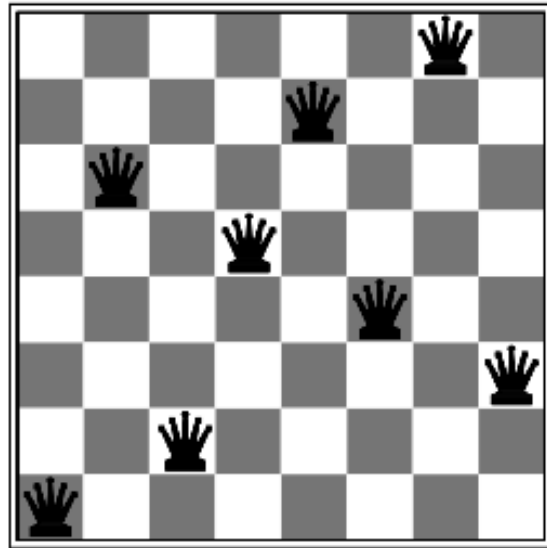


16257483

Esempio 8 regine

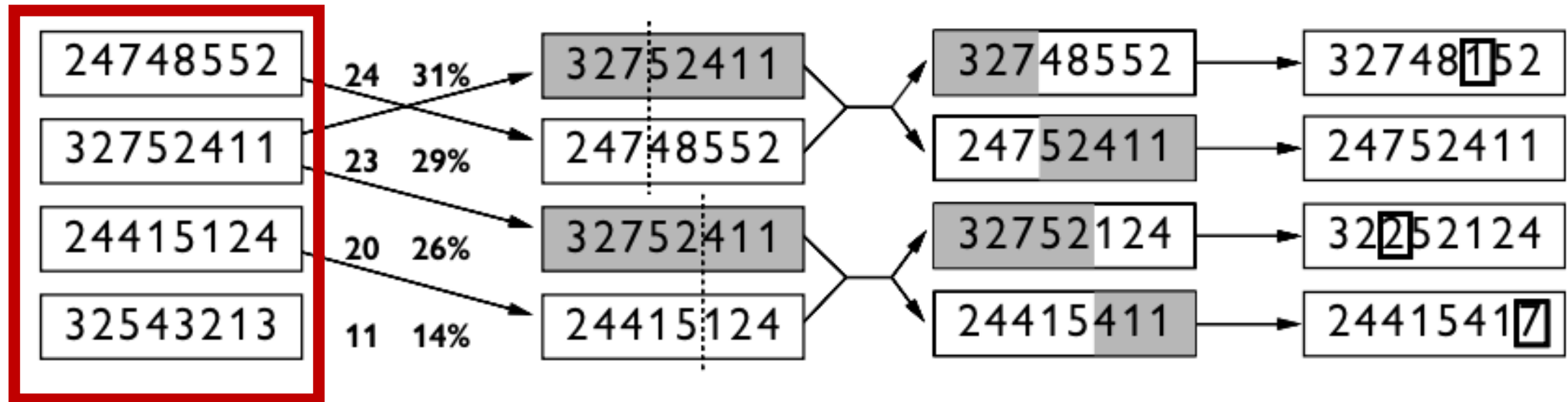
Fitness: valori migliori devono corrispondere a stati (configurazioni) migliori.

- numero di coppie regine che non si attaccano
 - valore = 28 per una soluzione



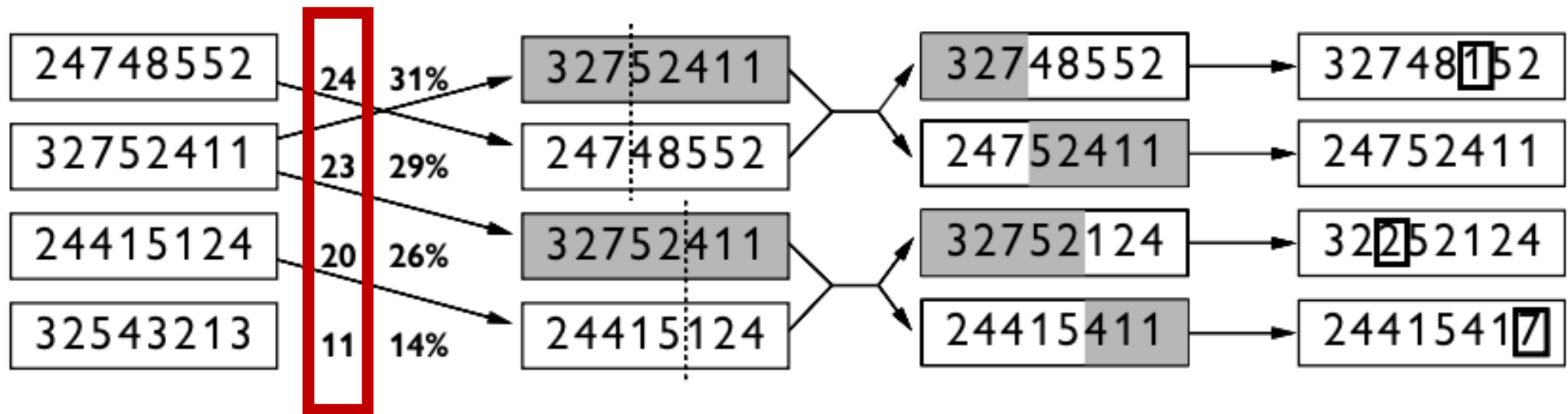
Valore di fitness = 27

Esempio 8 regine



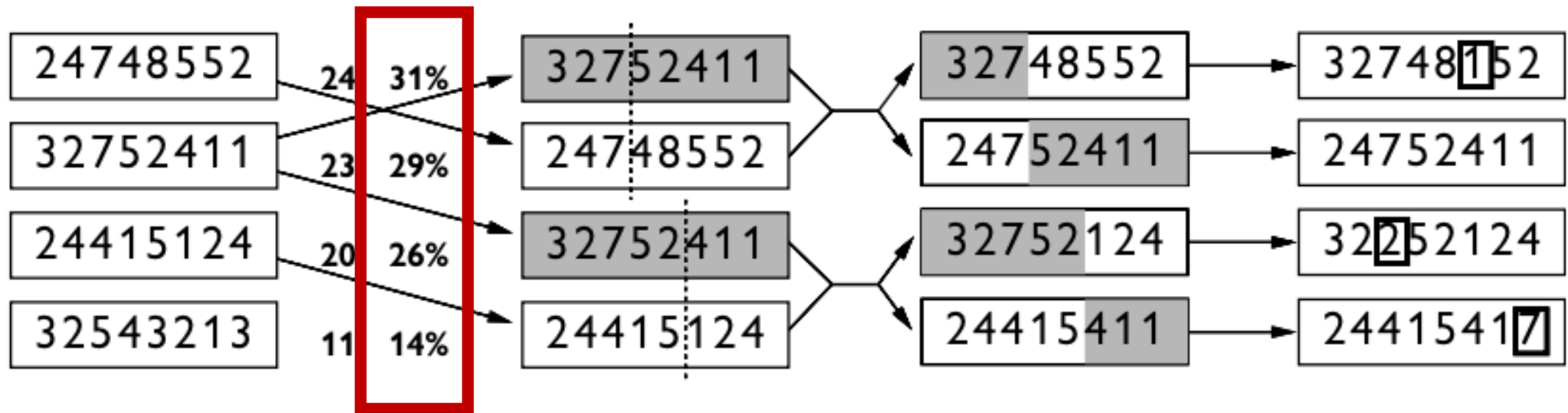
La popolazione iniziale

Esempio 8 regine



Calcolo del valore di fitness.
Le coppie vengono ordinate
per fitness decrescente.

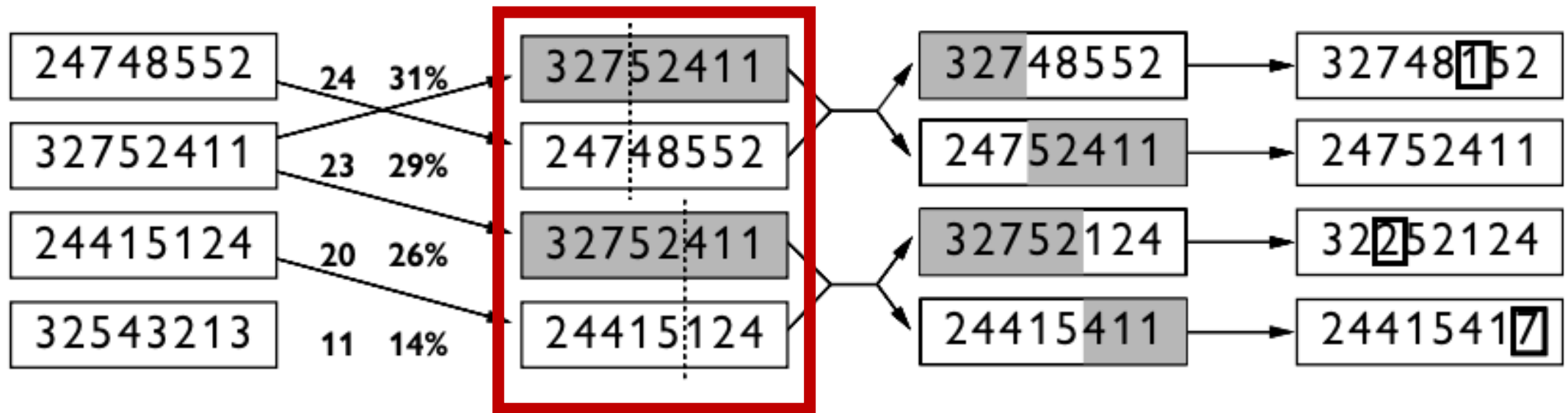
Esempio 8 regine



Calcolo del valore di fitness.
Le coppie vengono ordinate per
fitness decrescente.

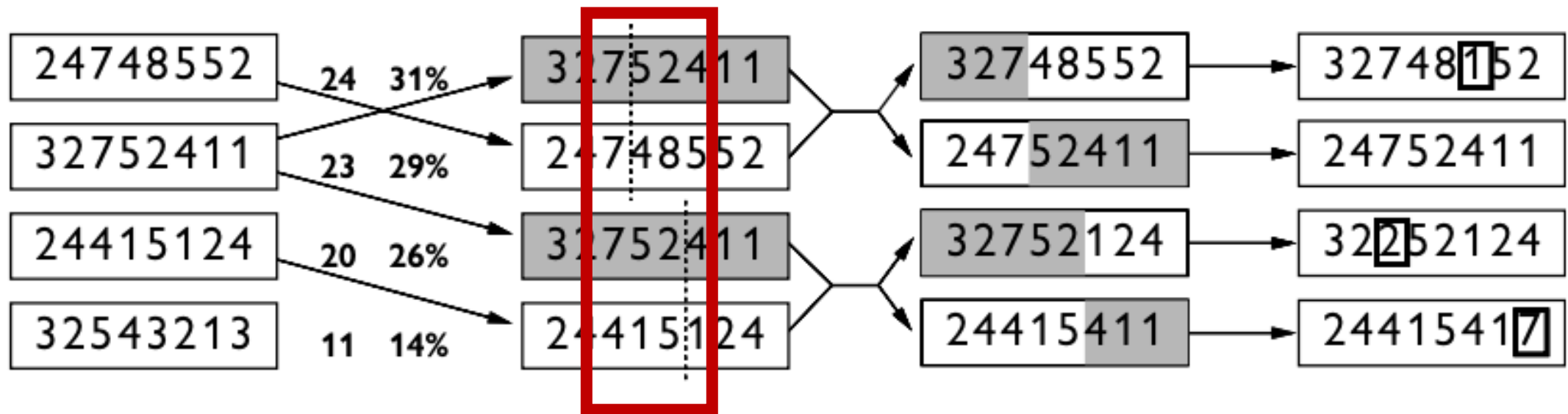
Valori normalizzati come
probabilità

Esempio 8 regine



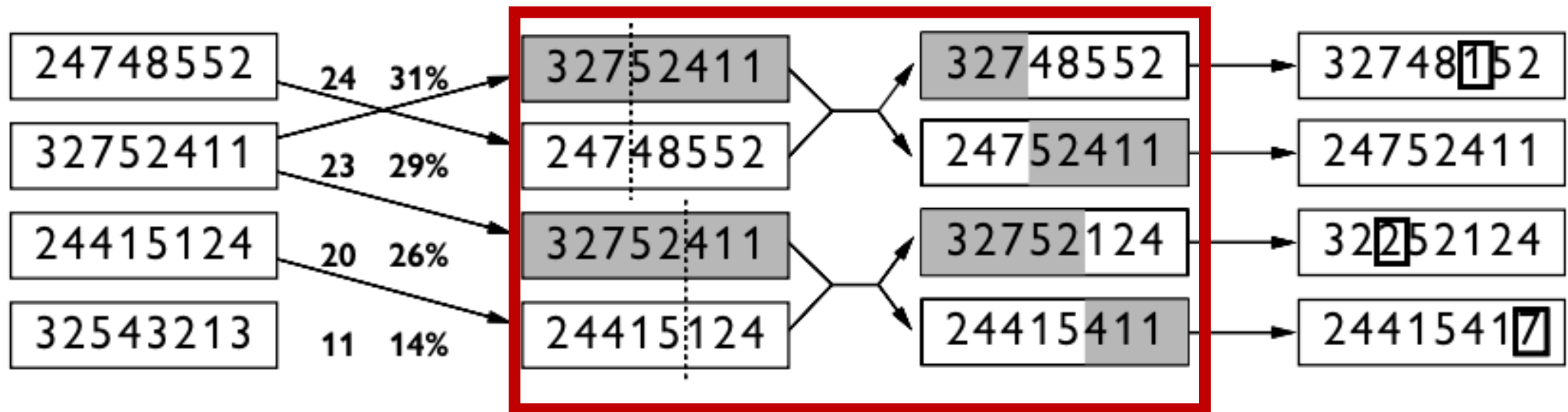
Selezionate due coppie di genitori, in base alle probabilità.

Esempio 8 regine



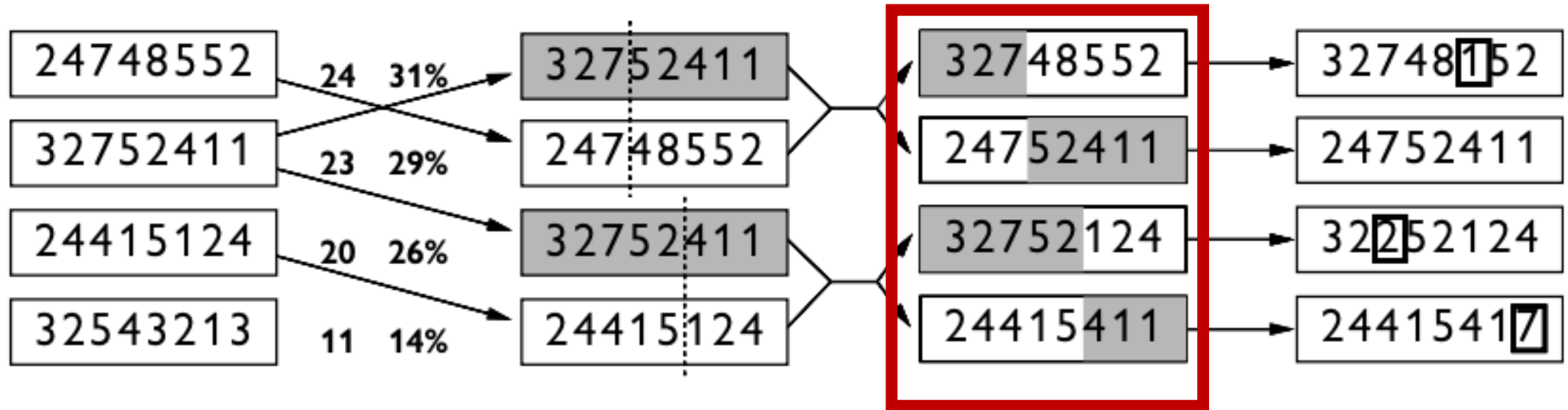
Per ogni coppia viene
selezionato casualmente un
punto di crossover

Esempio 8 regine



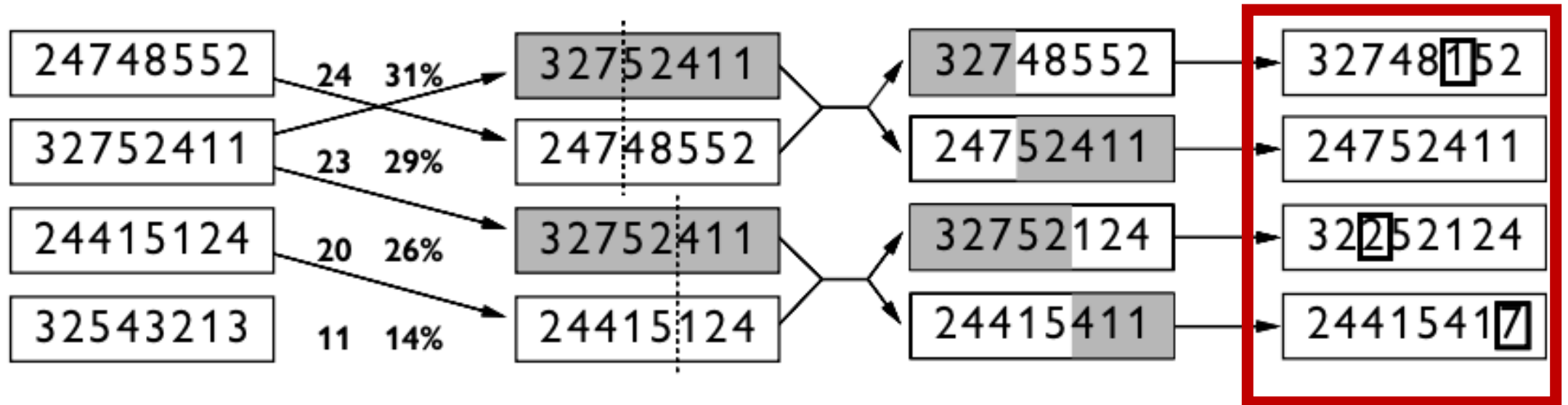
Le stringhe dei genitori
vengono incrociate nei punti di
crossover

Esempio 8 regine



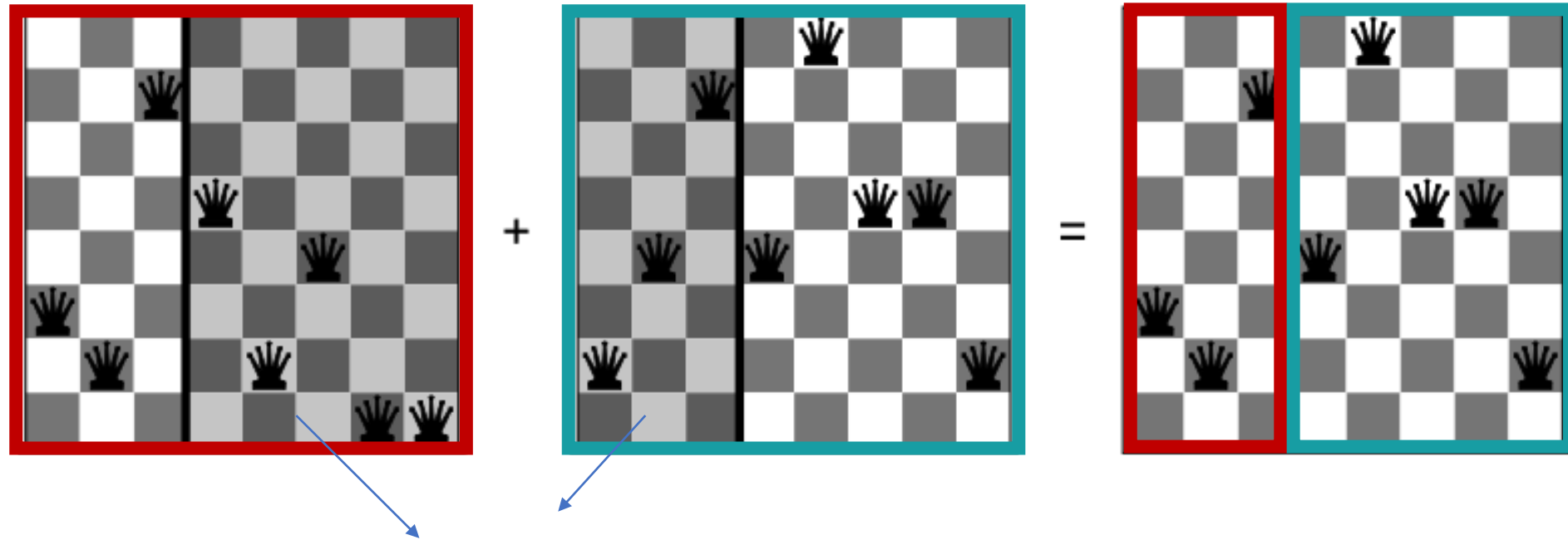
Viene ottenuta la
nuova generazione

Esempio 8 regine



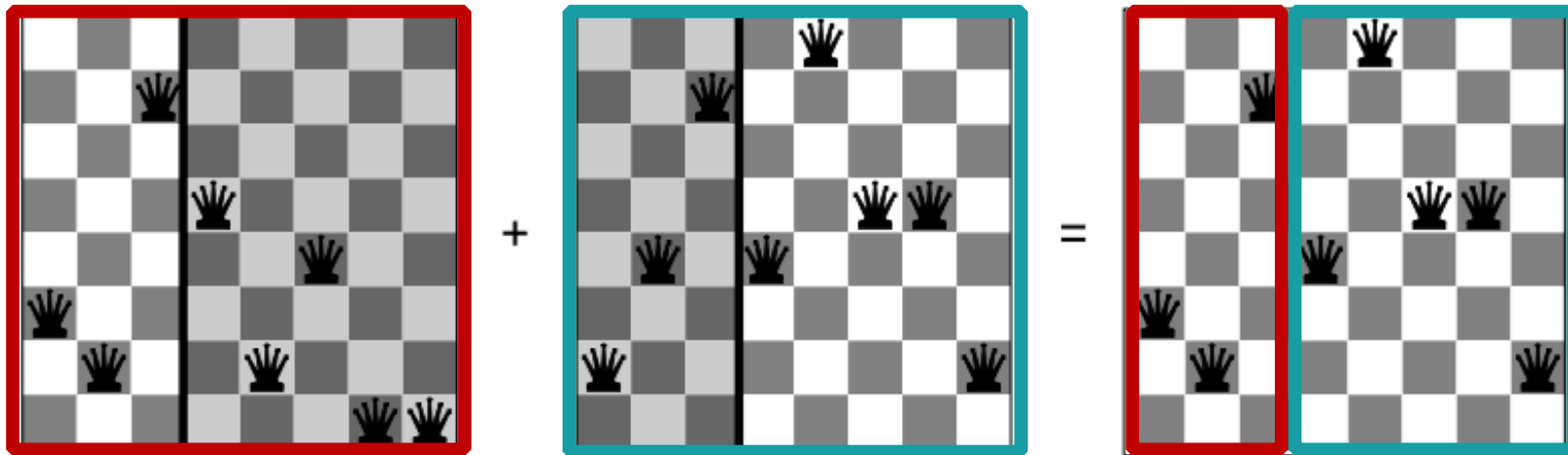
Mutazione
casuale con una
piccola
probabilità

Esempio 8 regine: nascita di un figlio



Perse nel processo di
crossover

Esempio 8 regine: nascita di un figlio



- Se i genitori sono molto diversi anche i nuovi stati sono diversi
- All'inizio spostamenti maggiori che poi si raffinano

Algoritmo genetico

function ALGORITMO-GENETICO(*popolazione*, *fitness*) **returns** un individuo

repeat

pesi $\leftarrow$ PESATO-DA(*popolazione*, *fitness*)

popolazione2 $\leftarrow$ lista vuota

for *i* = 1 **to** DIMENSIONE(*popolazione*) **do**

genitore1, *genitore2* $\leftarrow$ SELEZIONE-CASUALE-PESATA(*popolazione*, *pesi*, 2)

figlio $\leftarrow$ RIPRODUZIONE(*genitore1*, *genitore2*)

if (piccola probabilità casuale) **then** *figlio* $\leftarrow$ MUTAZIONE(*figlio*)

 aggiungi *figlio* a *popolazione2*

popolazione $\leftarrow$ *popolazione2*

until qualche individuo ha un valore della fitness sufficientemente alto, o è passato abbastanza tempo

return l'individuo migliore nella *popolazione* in base alla *fitness*

function RIPRODUZIONE(*genitore1*, *genitore2*) **returns** un individuo

n $\leftarrow$ LUNGHEZZA(*genitore1*)

c $\leftarrow$ numero casuale fra 1 e *n*

return CONCATENA(SOTTOSTRINGA(*genitore1*, 1, *c*), SOTTOSTRINGA(*genitore2*, *c* + 1, *n*))

Algoritmi genetici

Suggestivi (area del *Natural computing*: e.g. *swarm*, ...)



- Usati in molti problemi reali e.g. configurazione di circuiti e scheduling di lavori
- Funziona meglio se il problema (soluzioni) ha componenti significative rappresentate in sottostringhe
- **Punto critico:** la rappresentazione del problema in stringhe

Ricerca locale in spazi continui

Spazi continui

Molti casi reali hanno spazi di ricerca continua

- e.g. fondamentale per Machine Learning

Stato descritto da variabili continue, $x_1, \dots, x_n$

- vettore $\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$

Spazi continui

Prendiamo ad esempio movimenti in uno spazio 3D, con posizione data da

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

Un ambiente organizzato in spazi di stati continui può apparire **ostico**.

- **Fattori di ramificazione infiniti con gli approcci visti finora** (es., è possibile spostarsi in un punto qualsiasi dello spazio intorno ad ogni valore di $\mathbf{x}$ dato)

Spazi continui

Prendiamo ad esempio movimenti in uno spazio 3D, con posizione data da

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

In realtà esistono molti strumenti matematici per spazi continui, che portano ad approcci anche molto efficienti....

Spazi continui: approcci

■ Discretizzare

- ogni posizione è limitata a punti fissi in una griglia rettangolare
- possibile usare algoritmi di ricerca locale nello spazio discreto ottenuto

■ Gradiente empirico

- rendere limitato il fattore di ramificazione mediante un campionamento casuale degli stati successivi
- e.g., valuto $f(\mathbf{x} + \delta)$ e verifico se il valore ottenuto è migliore rispetto a quello di $f(\mathbf{x})$.

Spazi continui: gradiente

Se la f è continua e differenziabile, e.g. quadratica rispetto a $\mathbf{x}$ (vettore):

Il minimo o massimo si può cercare utilizzando il **gradiente**, che restituisce la direzione di massima pendenza nel punto

Spazi continui: gradiente

Data f obiettivo su 3D, il gradiente ∇f fornisce l'entità e la direzione della pendenza più ripida.

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \frac{\partial f}{\partial x_3} \end{bmatrix}$$

Spazi continui: gradiente

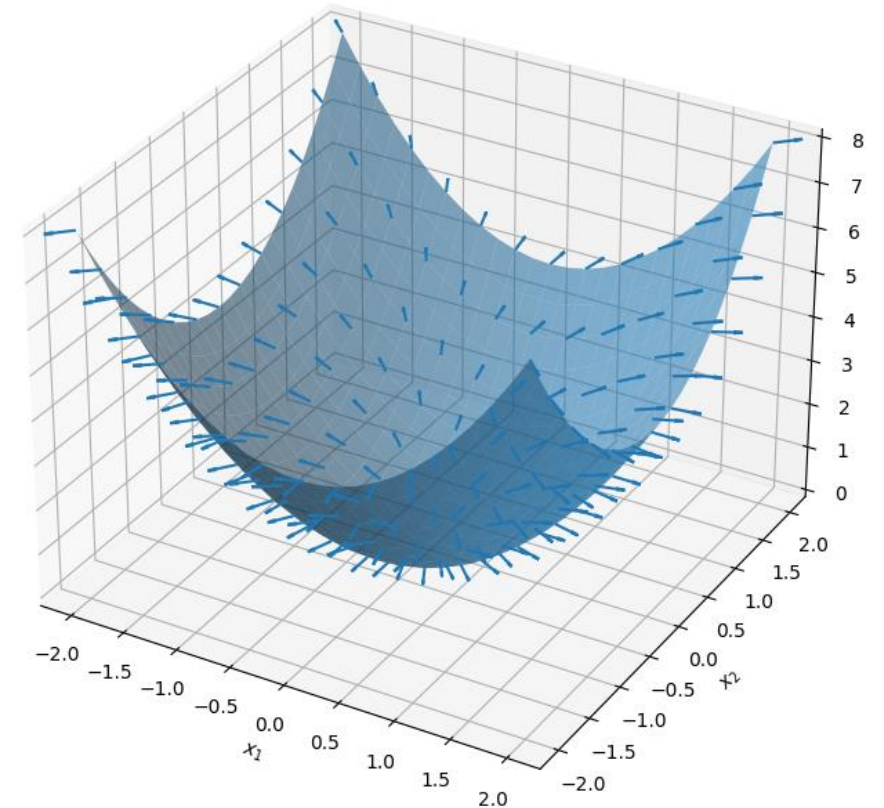
Data f obiettivo su 3D, il gradiente ∇f fornisce l'entità e la direzione della pendenza più ripida.

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \frac{\partial f}{\partial x_3} \end{bmatrix}$$

Le linee blu
indicano il vettore
del gradiente

Esempio: funzione di due sole variabili

Surface $f(x_1, x_2) = x_1^2 + x_2^2$ and gradient field $\nabla f = (2x_1, 2x_2)$



Spazi continui: gradiente

Usiamo hill climbing seguendo la massima pendenza.

Aggiorniamo lo stato in base alla formula:

Per massimizzare:
Andiamo nella direzione del
valore *crescente* di f

$$\mathbf{x}_{new} = \mathbf{x} + \eta \nabla f(\mathbf{x})$$

gradiente calcolato in $\mathbf{x}$

Step size

(e.g., costante positiva < 1)

Spazi continui: gradiente

Usiamo hill climbing seguendo la massima pendenza.

Aggiorniamo lo stato in base alla formula:

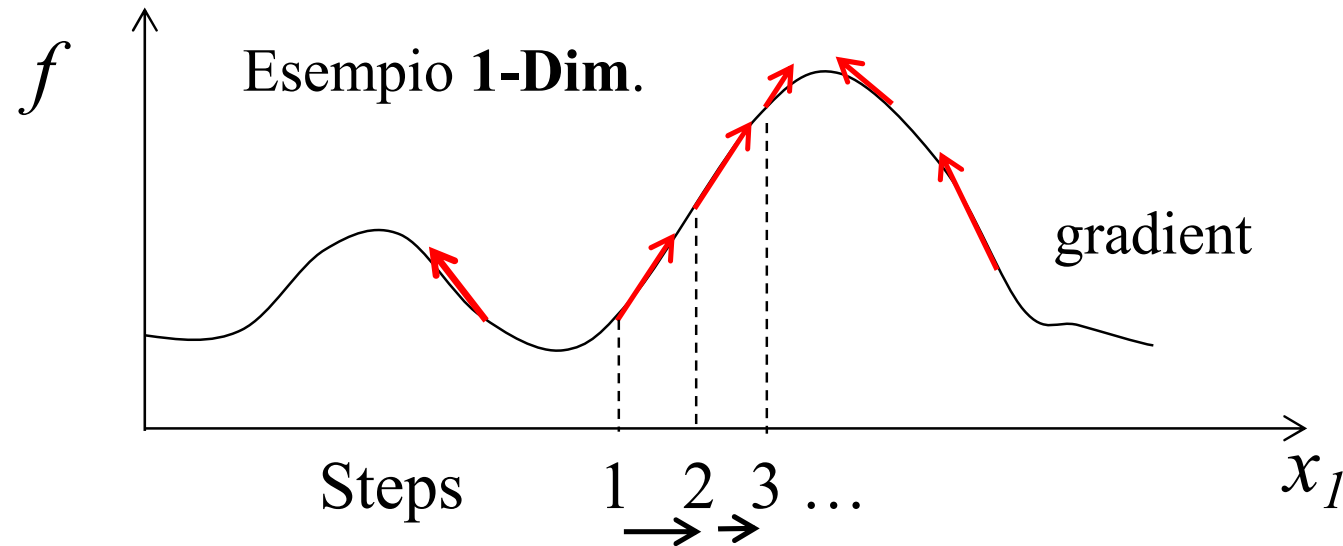
Per minimizzare:

Andiamo nella direzione del
valore *decrescente* di f

$$\mathbf{x}_{new} = \mathbf{x} - \eta \nabla f(\mathbf{x})$$

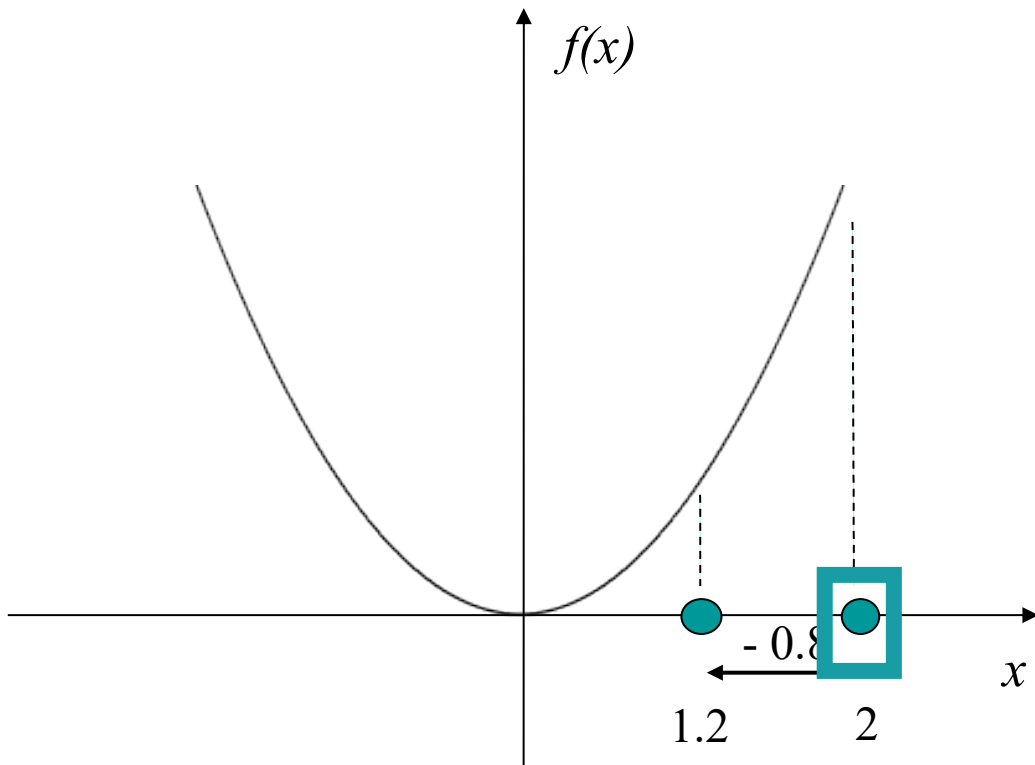
Quantifica la direzione dello
spostamento, senza cercarlo tra gli
infiniti possibili successori

Spazi continui: gradiente (esempio)



Nota generale: non sempre è necessario il
min/max assoluto: vedrete in parte III del corso
(Machine Learning)

Spazi continui: gradiente (esempio discesa verso il minimo)



$$f(x) = x^2 \quad \frac{df}{dx} = f'(x) = 2x$$

$$\mathbf{x}_{new} = \mathbf{x} - \eta \nabla f(\mathbf{x})$$

multidimensionale

$$x_{new} = x - \eta f'(x)$$

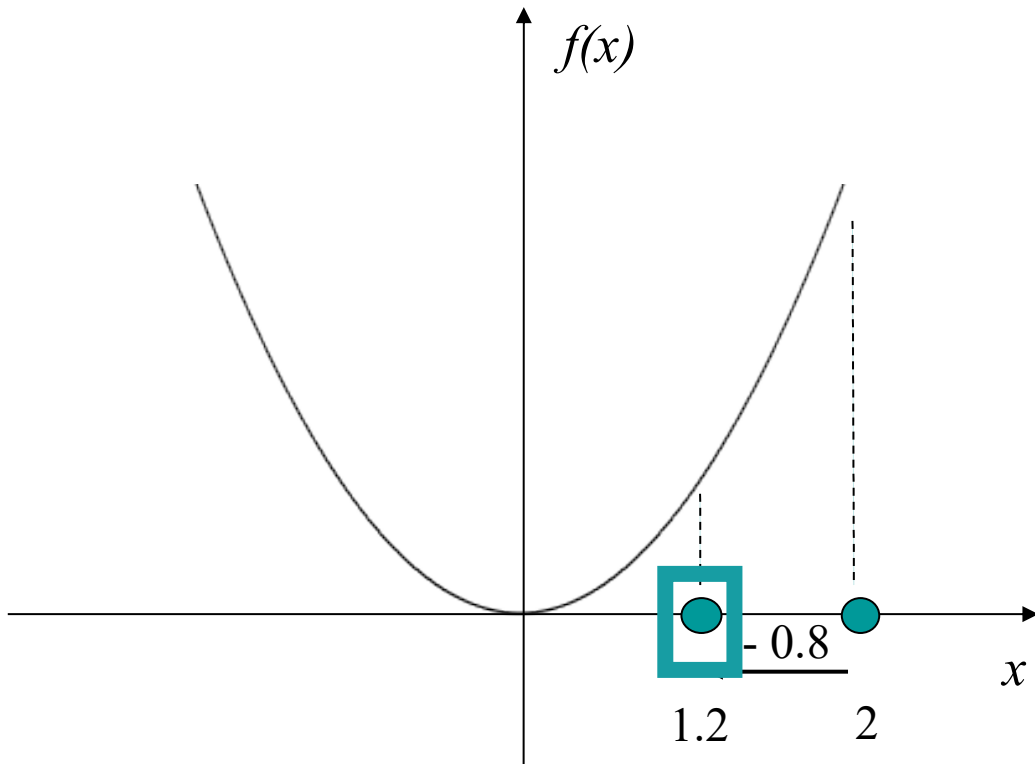
mono-dimensionale

$$\eta = 0.2$$

$$x = 2 \quad f'(2) = 4$$

$$x_{new} = x - \eta f'(x) = 2 - 0.2 \times 4 = 1.2$$

Spazi continui: gradiente (esempio discesa verso il minimo)



$$f(x) = x^2 \quad \frac{df}{dx} = f'(x) = 2x$$

$$\mathbf{x}_{new} = \mathbf{x} - \eta \nabla f(\mathbf{x})$$

multidimensionale

$$x_{new} = x - \eta f'(x)$$

mono-dimensionale

$$\eta = 0.2$$

$$x = 2 \quad f'(2) = 4$$

$$x_{new} = x - \eta f'(x) = 2 - 0.2 \times 4 = 1.2$$

Ricerca con azioni non
deterministiche

Ambienti più realistici

Gli agenti *risolutori di problemi* “classici” assumono:

- Ambienti **completamente osservabili**
- Azioni/ambienti **deterministici**
- Il piano generato è una **sequenza di azioni** che può essere generato *offline* e **eseguito senza imprevisti**
- Le **percezioni** non servono se non nello stato iniziale

Soluzioni più complesse

- In un ambiente parzialmente osservabile e non deterministico le **percezioni** sono importanti
 - restringono gli stati possibili
 - informano sull'effetto dell'azione
- Più che un *piano* l'agente può elaborare una “*strategia*”, che tiene conto delle diverse eventualità: un **piano con contigenza**

Rilassiamo il vincolo di determinismo

Quando l'ambiente è non deterministico, l'agente non sa in quale stato arriverà dopo l'esecuzione di un'azione.

- **determinismo**: “sono nello stato s_1 e se eseguo l'azione a andrò nello stato s_2 ”
- **non determinismo**: “Sono nello stato s_1 o s_3 , e se eseguo l'azione a passerò nello stato s_2 , s_4 o s_5 ”
- **stato-credenza**: un insieme di stati che l'agente ritiene siano possibili

Il mondo dell'aspirapolvere erratico

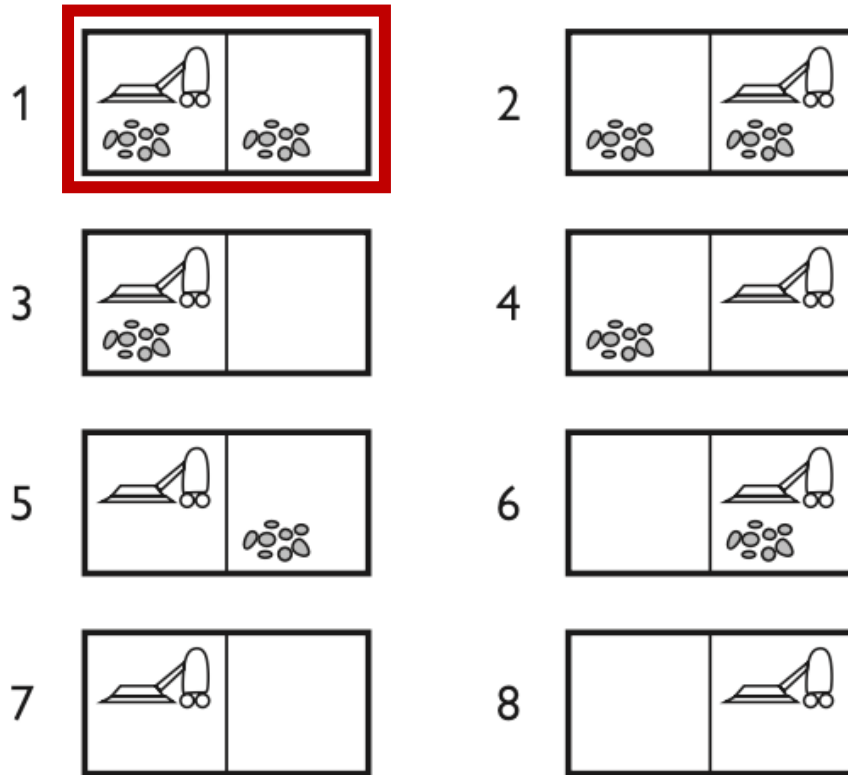
Introduciamo non determinismo.

L'azione *Aspira* opera come segue:

- applicata in un riquadro sporco, lo pulisce, e talvolta pulisce anche un riquadro adiacente
- applicata ad un riquadro pulito, talvolta deposita dello sporco

Il mondo dell'aspirapolvere erratico

Se iniziamo qui, nessuna singola sequenza di azioni risolve il problema



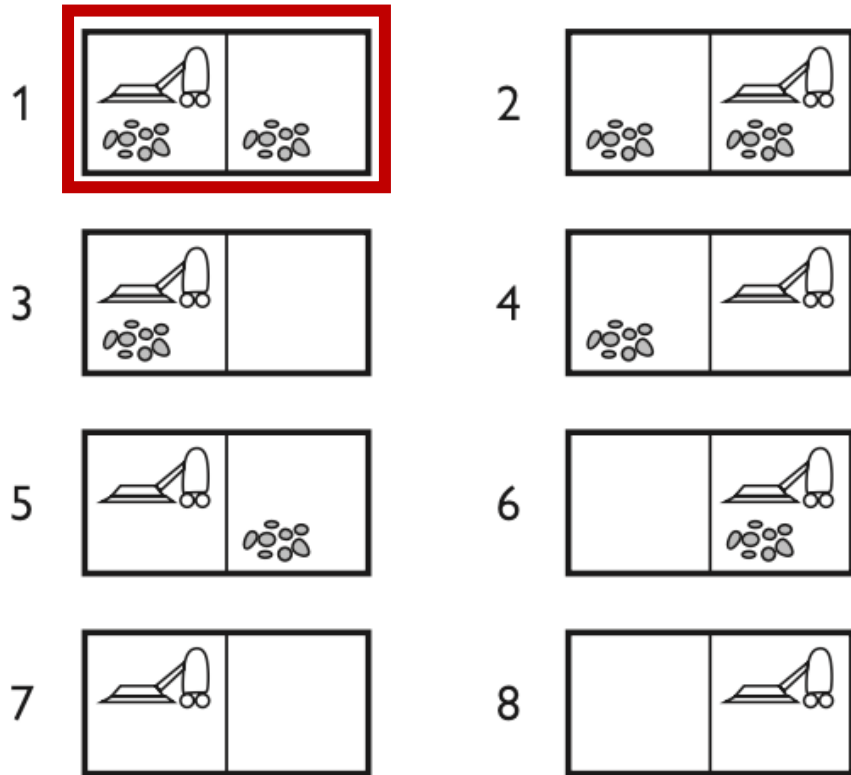
Invece di definire il modello di transizione con una funzione **RISULTATO** (che restituisce un singolo stato)

■ Definiamo una funzione

RISULTATI

- restituisce un insieme di possibili stati risultato
- es., $\text{RISULTATI}(1, \text{Aspira}) = \{5, 7\}$

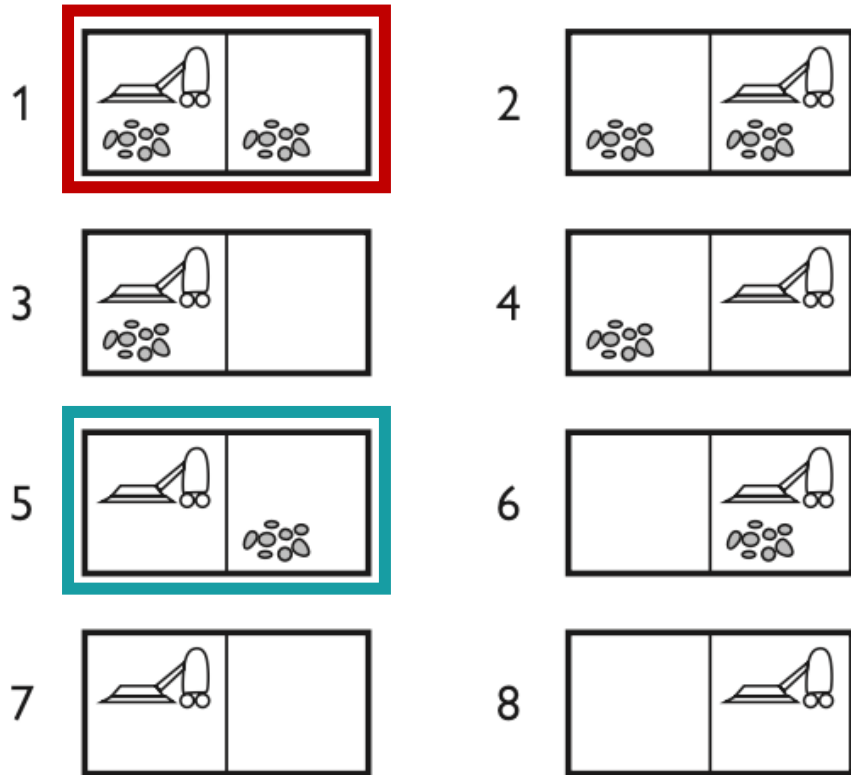
Il mondo dell'aspirapolvere erratico



Usiamo invece un **piano condizionale**:

```
[Aspira,  
  if Stato = 5  
    then [Destra, Aspira]  
    else []  
]
```

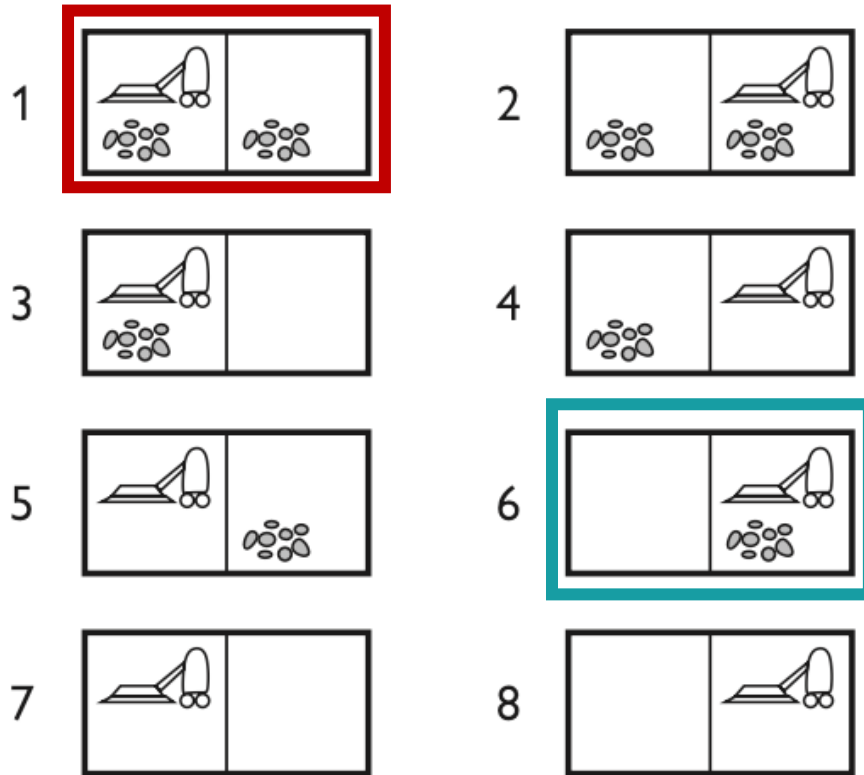
Il mondo dell'aspirapolvere erratico



Usiamo invece un **piano condizionale**:

```
[Aspira,  
  if Stato = 5  
    then [Destra, Aspira]  
    else []  
]
```

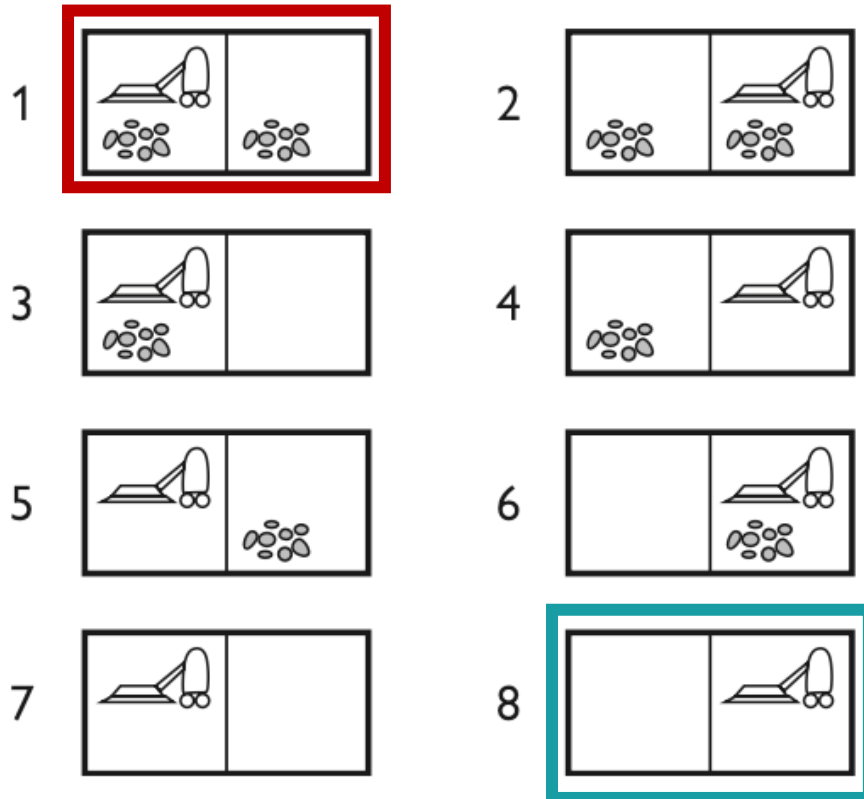
Il mondo dell'aspirapolvere erratico



Usiamo invece un **piano condizionale**:

```
[Aspira,  
  if Stato = 5  
    then [Destra, Aspira]  
    else []  
]
```

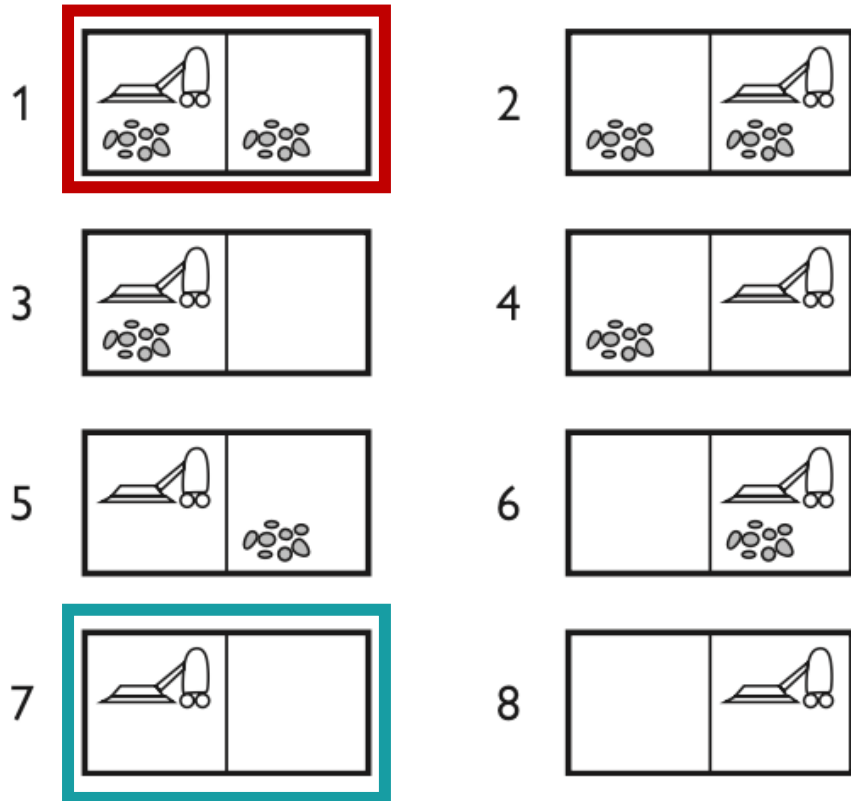
Il mondo dell'aspirapolvere erratico



Usiamo invece un **piano condizionale**:

```
[Aspira,  
  if Stato = 5  
    then [Destra, Aspira]  
    else []  
]
```

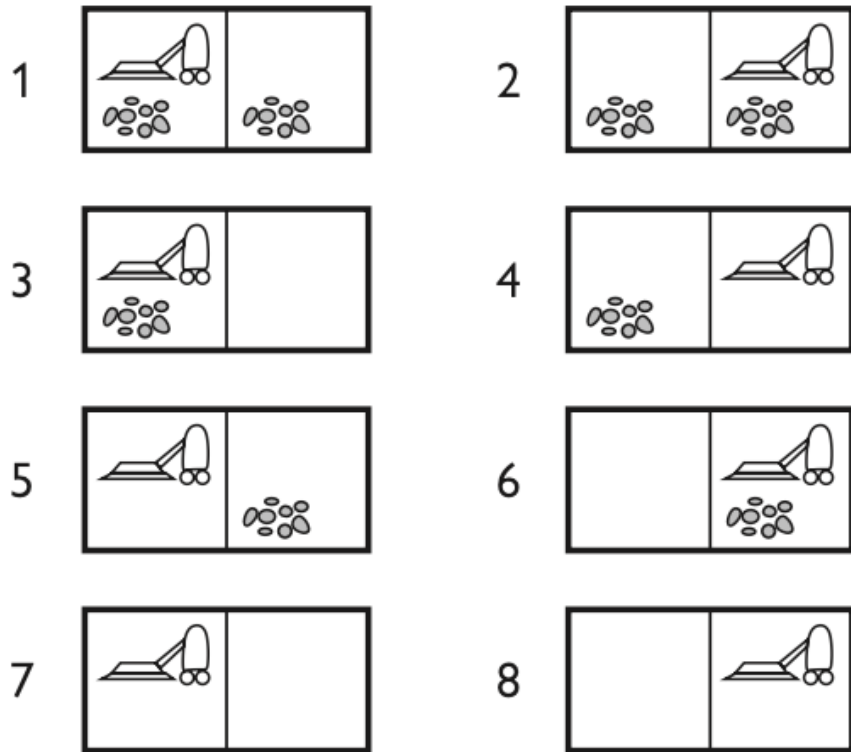
Il mondo dell'aspirapolvere erratico



Usiamo invece un **piano condizionale**:

```
[Aspira,  
  if Stato = 5  
    then [Destra, Aspira]  
    else []  
]
```

Il mondo dell'aspirapolvere erratico



Un piano condizionale può contenere passi if-then-else.

Le soluzioni sono **alberi** e non **sequenze**.

Come troviamo soluzioni contingenti?

Alberi di ricerca AND-OR

Alberi di ricerca AND-OR

- Nodo **OR**: le scelte dell'agente
 - l'agente deve compiere una scelta di 1 sola azione
- Nodo **AND**: le diverse contingenze
 - più stati possibili, da considerare tutti

Alberi di ricerca AND-OR

- Nodo OR: le scelte dell'agente
 - l'agente deve compiere una scelta di 1 sola azione
- Nodo AND: le diverse contingenze
 - più stati possibili, da considerare tutti

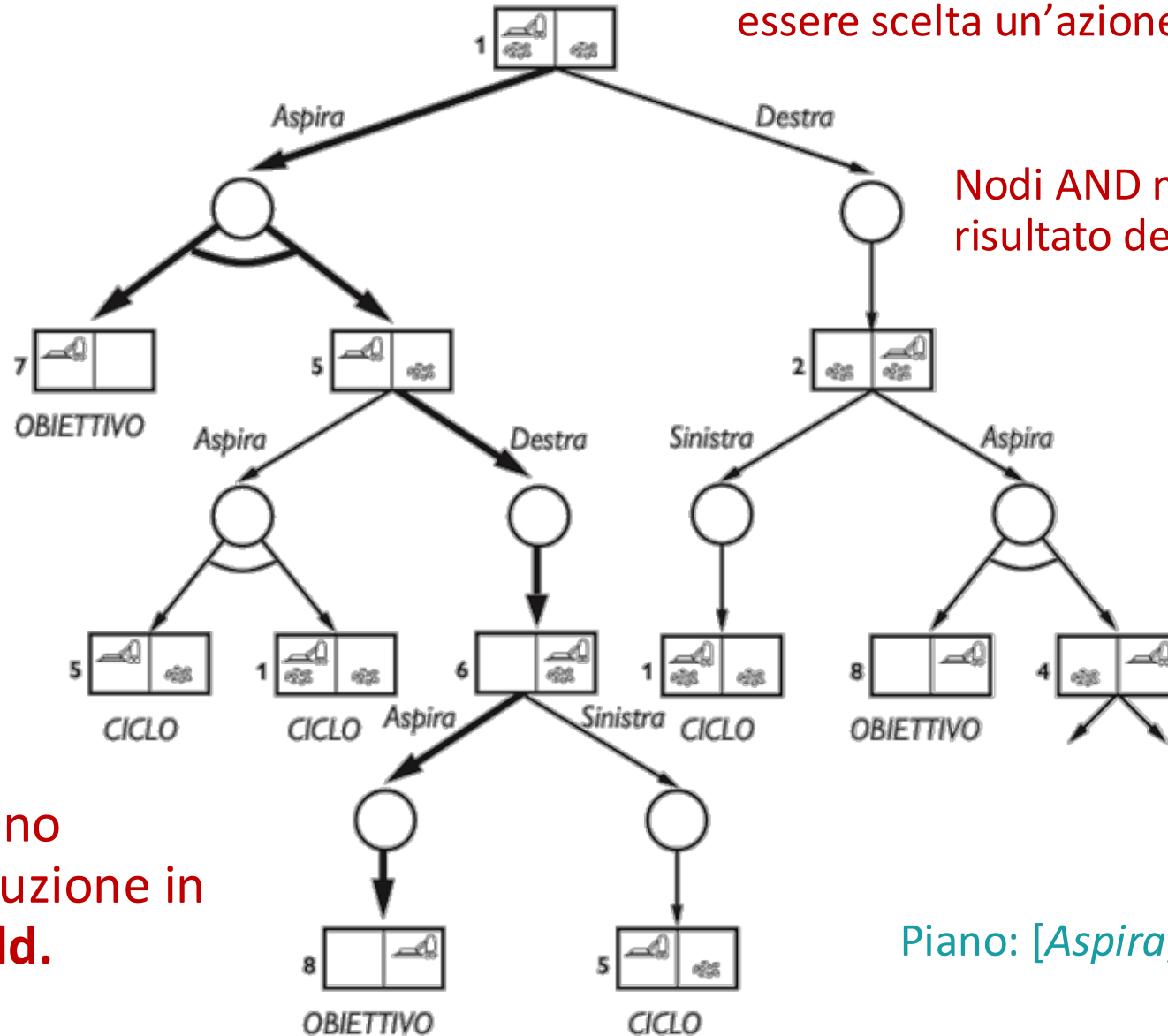
Una soluzione a un problema di ricerca AND-OR è un albero che:

- ha un nodo obiettivo in ogni foglia
- specifica una sola azione nei nodi OR
- include tutti gli archi uscenti da nodi AND
 - tutte le contingenze

I nodi degli stati sono nodi OR in cui deve essere scelta un'azione.

Nodi AND mostrati come cerchi. Ogni risultato deve essere gestito

Esempio



Piano
soluzione in
bold.

Piano: *[Aspira, if Stato=5 then [Destra, Aspira] else []]*

Stop (IIA 6 CFU)

Il resto di questa presentazione vi resta disponibile qui di seguito per consultazione dal materiale aa 2023/24

Credits: Prof. Alessio Micheli

STOP per IIA 6 crediti

Fin qui: ALMA ed. III fino a cap 4.3 incluso

ALMA ed. IV fino a 4.3 incluso

Il resto di questa presentazione vi resta
disponibile per consultazione

Algoritmo ricerca grafi AND-OR

function Ricerca-Grafo-AND-OR (*problema*)

returns un piano condizionale oppure *fallimento*

Ricerca-OR(*problema.StatoIniziale*, *problema*, [])

function Ricerca-OR(*stato*, *problema*, *cammino*) // nodi OR

returns un piano condizionale oppure *fallimento*

If *problema.TestObiettivo*(*stato*) **then return** [] // piano vuoto

If *stato* è su *cammino* **then return** *fallimento* // spezza i cicli

for each *azione* **in** *problema.Azione*(*stato*) **do**

piano ← Ricerca-AND (*Risultati*(*stato*, *azione*), *problema*, [*stato*|*cammino*])

If *piano* ≠ *fallimento* **then return** [*azione* | *piano*]

return *fallimento*

Algoritmo ricerca grafi AND-OR

function Ricerca-AND(*stati*, *problema*, *cammino*) // nodi AND

returns un piano condizionale oppure *fallimento*

for each s_i **in** *stati* **do**

$piano_i \leftarrow$ Ricerca-OR(s_i , *problema*, *cammino*)

If $piano_i = fallimento$ **then return** *fallimento*

return

 [**if** s_1 **then** $piano_1$ **else**

if s_2 **then** $piano_2$ **else**

 ...

if s_{n-1} **then** $piano_{n-1}$ **else** $piano_n$]

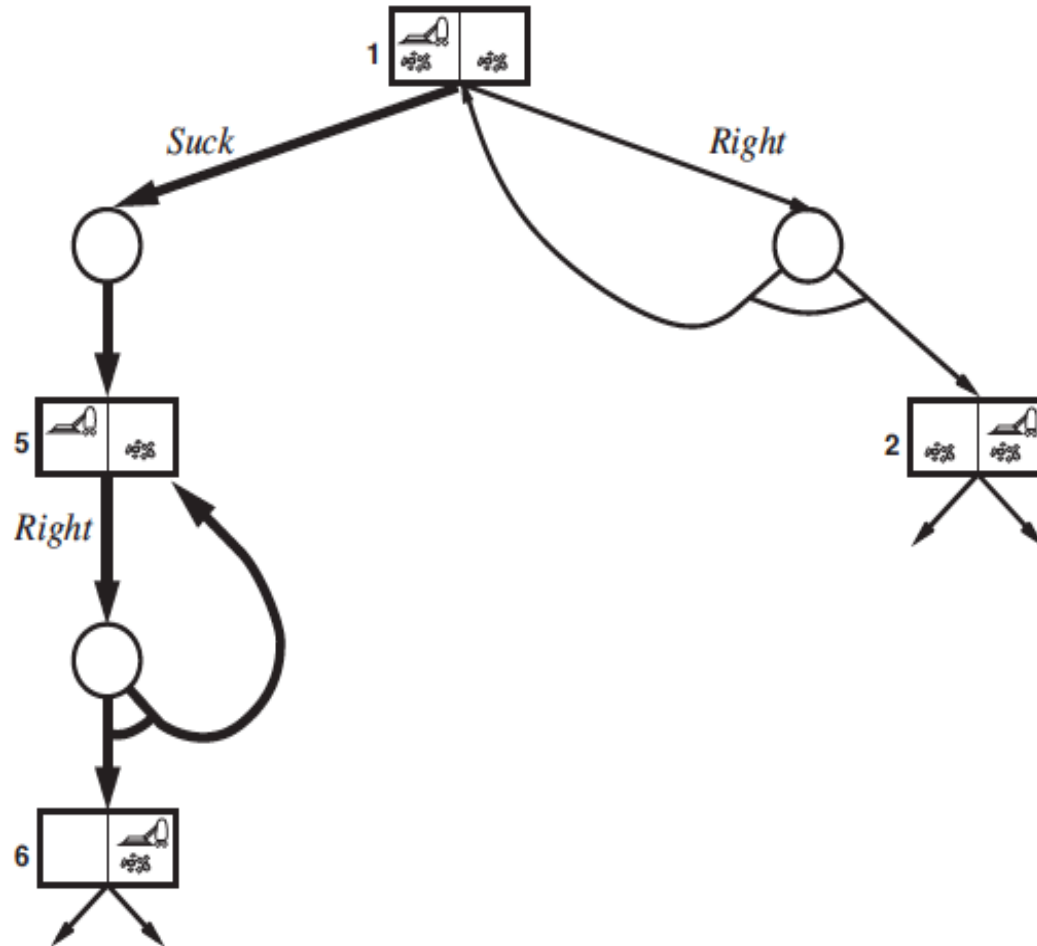
Ritorna un piano condizionale
Con azioni dei nodi OR

Ancora azioni non deterministiche

L'aspirapolvere slittante

- Comportamento:
 - *Quando si sposta può scivolare e rimanere nella stessa stanza*
 - Es. Risultati(Destra, 1) = {1, 2}
- Variazioni necessarie
 - Continuare a provare ... [finche' riesce ad andare a destra]
 - Il **piano di contingenza** potrà avere dei cicli

Aspirapolvere slittante: soluzione



Piano: [*Aspira*, L_1 : *Destra*, **if** Stato=5 **then** L_1 **else** *Aspira*]

Osservazione

- Bisogna distinguere tra:
 1. Osservabile e non deterministico (es. aspirapolvere slittante)
 2. Non (o parzialmente) osservabile e deterministico (es. non so se la chiave aprirà la porta)
- In questo secondo caso, se la chiave è sbagliata, si può provare all' infinito ma niente cambierà!

Ricerca con osservazioni parziali

- Le percezioni non sono sufficienti a determinare lo stato esatto, anche se l'ambiente è deterministico.
- **Stato credenza**: un insieme di *stati possibili* in base alle conoscenze dell'agente
- **Problemi senza sensori** (*sensorless* o **conformanti**)
- Si possono trovare soluzioni anche senza affidarsi ai sensori utilizzando stati-credenza

Ambiente non osservabile:

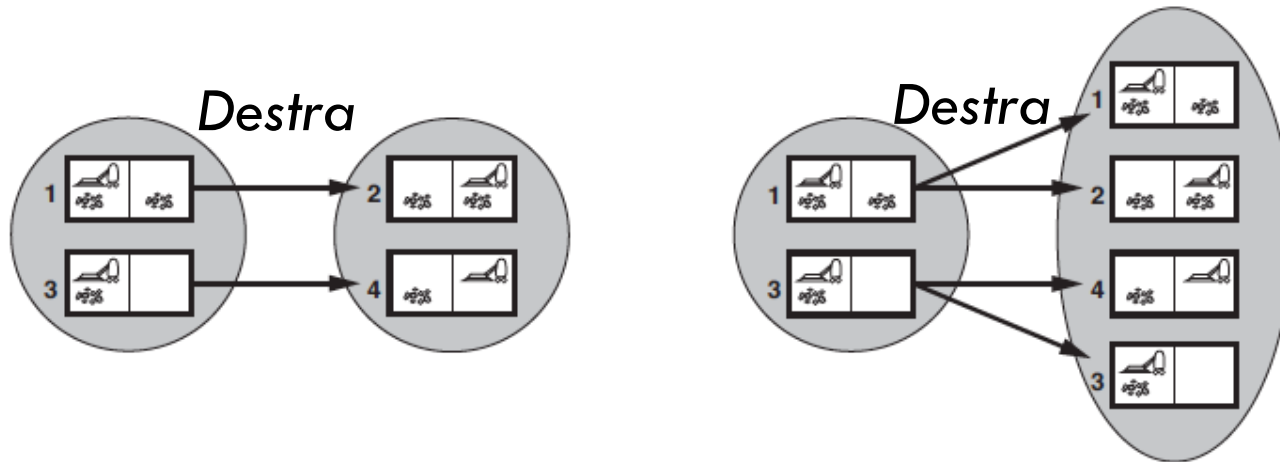
Aspirapolvere senza sensori

- L'aspirapolvere:
 - non percepisce la sua locazione, né se la stanza è sporca o pulita
 - conosce la geografia del suo mondo e l'effetto delle azioni
- Inizialmente tutti gli stati sono possibili
 - Stato iniziale = $\{1, 2, 3, 4, 5, 6, 7, 8\}$
- Le azioni riducono gli stati credenza
- Nota: nello spazio degli stati credenza l'ambiente è **osservabile** (l'agente conosce le sue credenze)

Formulazione di problemi con stati-credenza

- Se N numero stati , 2^N sono i possibili stati credenza
- **Stato-credenza iniziale** $SC_0 \subseteq$ insieme di tutti gli N stati
- **Azioni**(b) = unione delle azioni *lecite* negli stati in b (ma se azioni illecite in uno stato hanno effetti dannosi meglio intersezione)
- **Modello di transizione**: gli stati risultanti sono quelli ottenibili applicando le azioni a uno stato qualsiasi (l' unione degli stati ottenibili dai diversi stati possibili con le azioni eseguibili)

Problemi con stati-credenza (cnt.)

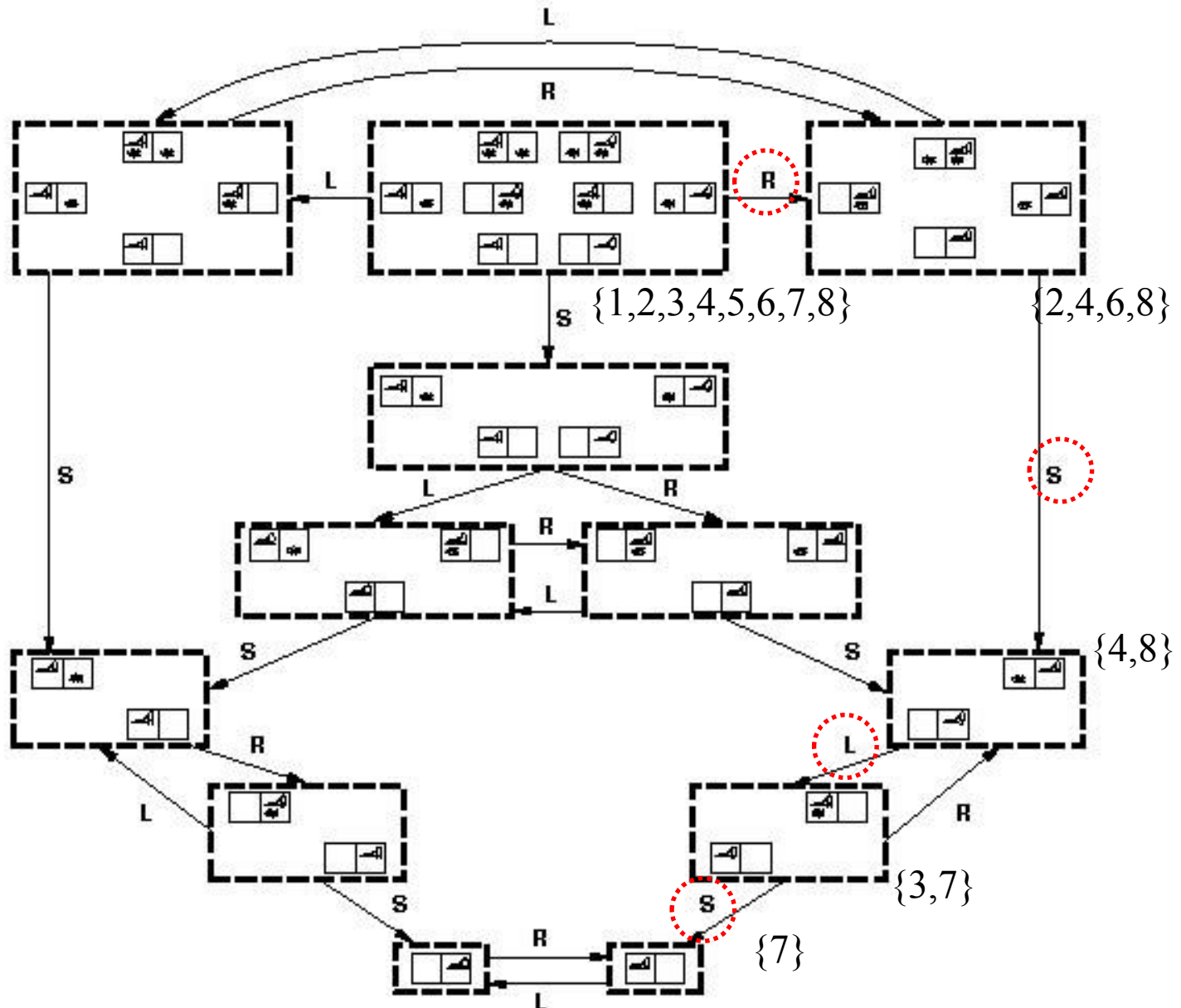


Senza sensori deterministico

Senza sensori e slittante (non det.)

- **Test obiettivo:** tutti gli stati nello stato credenza devono soddisfarlo
- **Costo di cammino:** il costo di eseguire un'azione potrebbe dipendere dallo stato, ma assumiamo di no

Il mondo dell'aspirapolvere senza sensori (determ.)



Ricerca: ottimizzazioni

- Si può effettuare un Ricerca-Grafo e controllare, generando s , se si è già incontrato uno stato credenza $s'=s$ e trascurare s
- Si può anche “potare” in modo più efficace in base al fatto che:
 - Se $s' \subseteq s$, allora ogni sequenza di azioni che è una soluzione per s lo è anche per s'
 - Se $s' \subseteq s$ (s' già incontrato) si può trascurare s
 - Se $s \subseteq s'$ e da s' si è trovata una soluzione si può trascurare s

Soluzione incrementale

- Dovendo trovare una soluzione per $\{1, 2, 3 \dots\}$ si cerca una soluzione per stato 1 e poi si controlla che funzioni per 2 e i successivi; se no se ne cerca un'altra per 1 ...
- Scopre presto i fallimenti ma cerca un'unica soluzione che va bene per tutti gli stati
- Non è una strategia completa ma è sicuramente più efficiente

Ricerca della soluzione

- Gli stati credenza possibili sono $2^8=256$ ma solo 12 sono raggiungibili
- In generale lo spazio di ogni stato può essere molto più grande con gli “stati credenza”
- La rappresentazione atomica obbliga a elencare tutti gli stati. Non è molto “compatta”. Non così con una rappresentazione più strutturata (lo vedremo)

Ricerca con osservazioni

- Ambiente parzialmente osservabile
- Esempio: *l'aspirapolvere con sensori **locali** che percepisce la sua posizione e lo sporco nella stanza in cui si trova (ma non nelle altre stanze)*
- Le percezioni diventano importanti
 - Assumiamo Percezioni(s)

Ricerca con osservazioni parziali

- Le percezioni assumono un ruolo
 - $\text{Percezioni}(s) = \text{null}$ in problemi *sensorless*
 - $\text{Percezioni}(s) = s$, ambienti osservabili
 - $\text{Percezioni}(s) = \text{percezioni [possibili] nello stato } s$
- Le percezioni restringono l'insieme di stati possibili
 - Esempio: [A, Sporco] percezione stato iniziale
Stato iniziale = $\{1, 3\}$

Il modello di transizione si complica

La transizione avviene in tre fasi:

1. Predizione dello stato credenza per effetto delle azioni:

$Predizione(b, a) = b'$

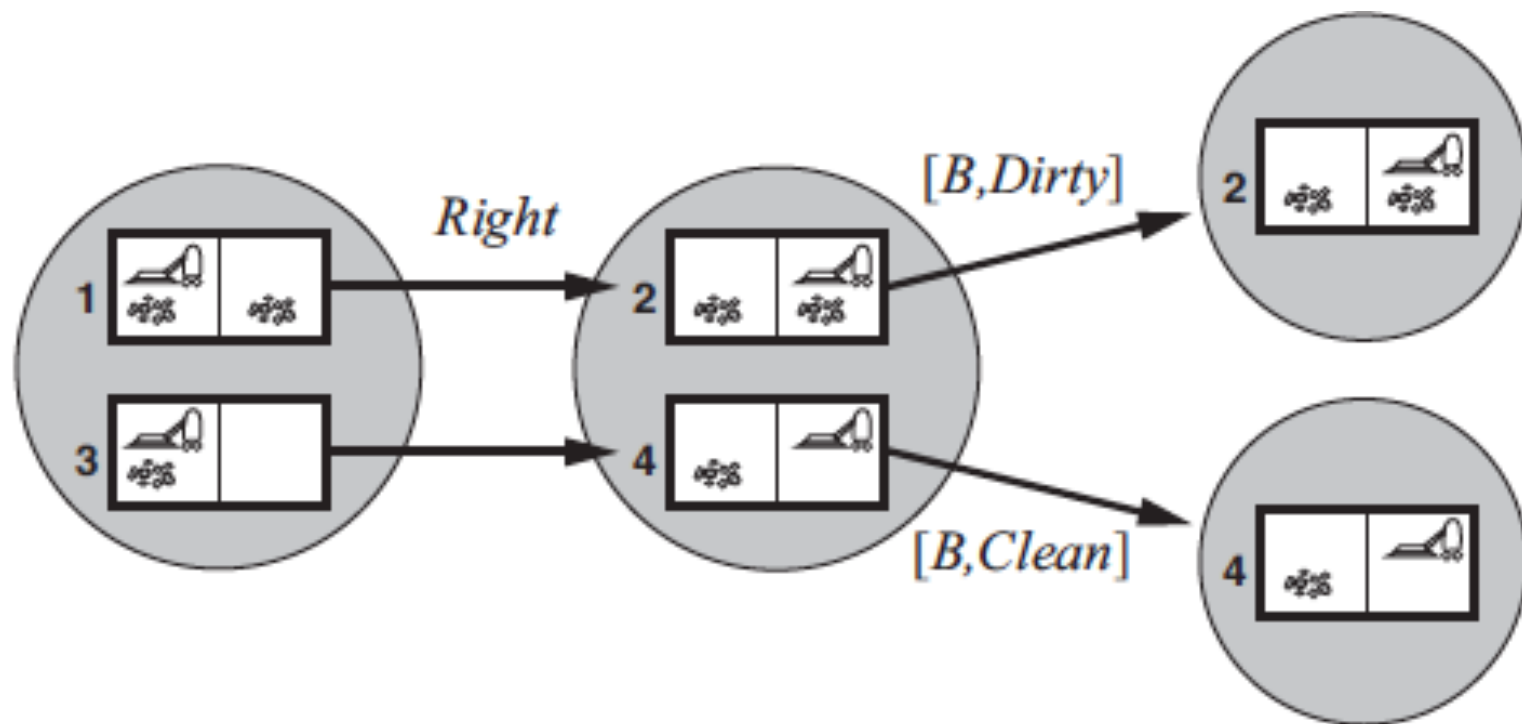
2. Predizione dell'osservazione: *Percezioni-possibili(b')*

3. Calcolo *aggiornamento* (insieme di stati credenza compatibili con lo stato credenza predetto e le possibili osservazioni):

$b'' = Aggiorna(Predizione(b, a), o)$

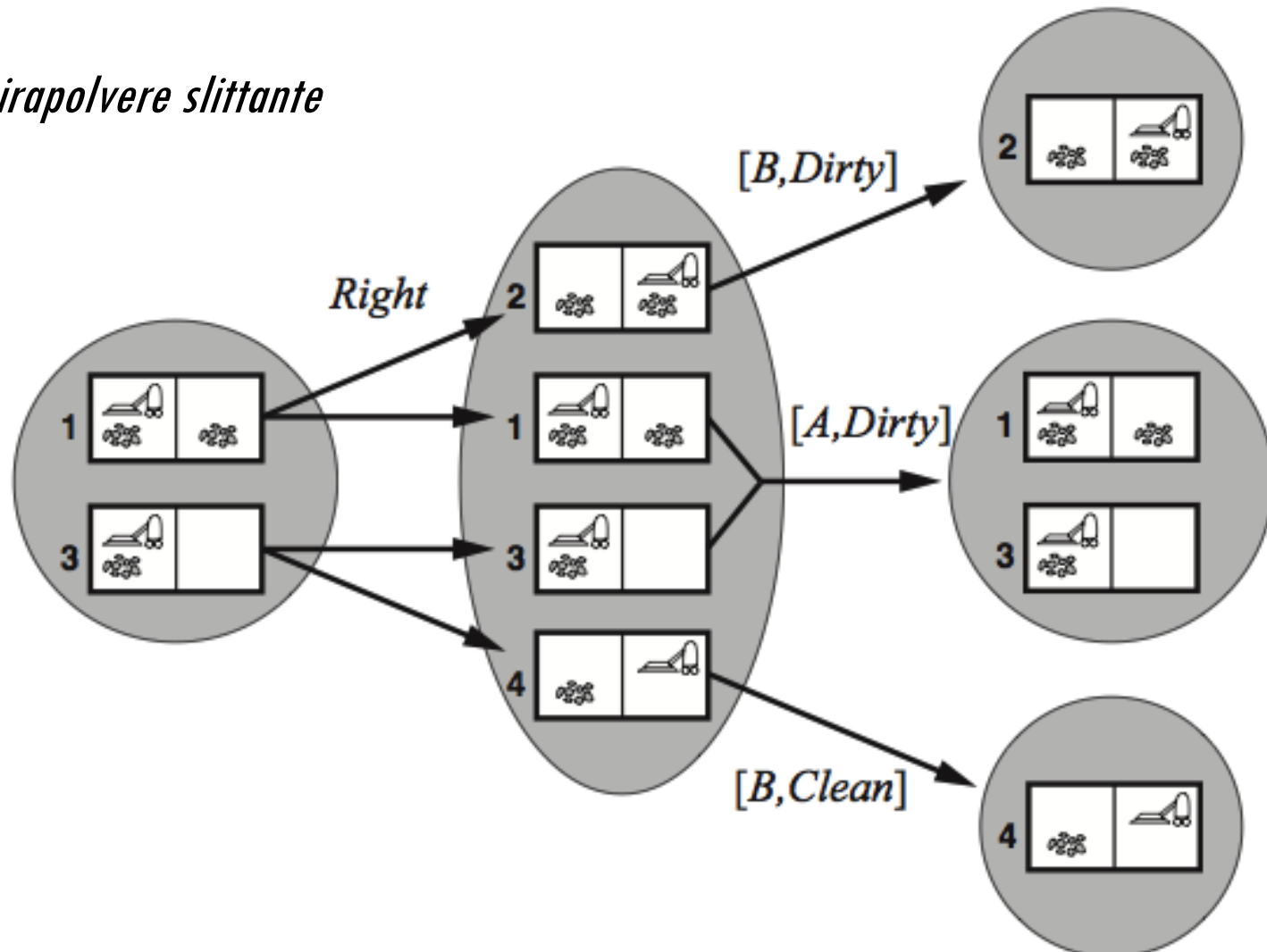
per ogni possibile osservazione o

Transizione con azioni deterministiche



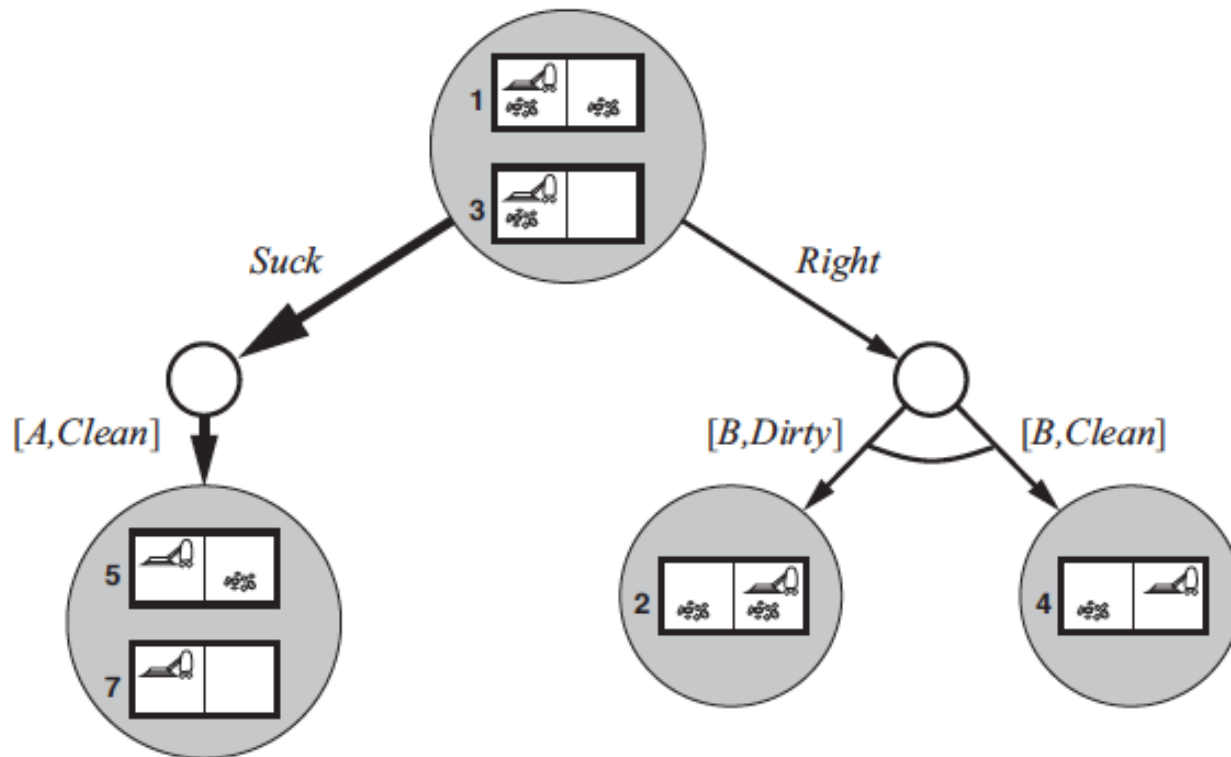
Transizione con azioni non deterministiche

Aspirapolvere slittante



Aspirapolvere con sensori locali

Per pianificare ci servono grafi AND-OR su stati credenza



[Aspira, Destra, if statoCredenza = {6} then Aspira else []]

Ricerca *online*

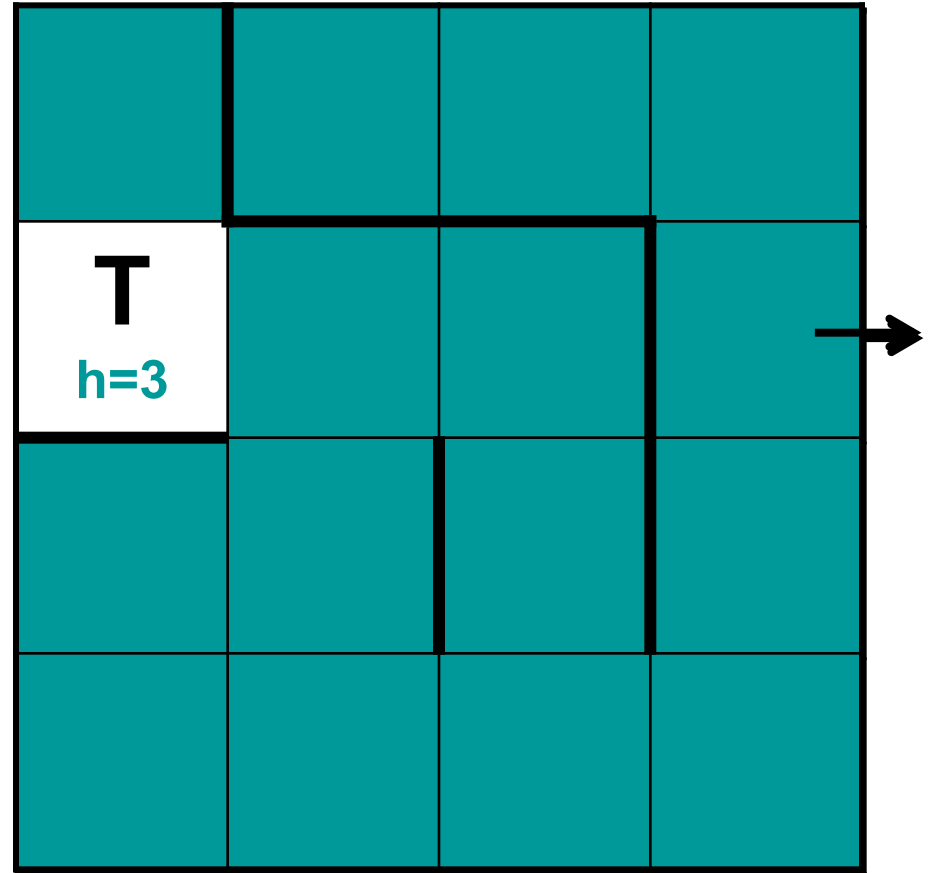
- Ricerca *offline* e ricerca *online*
- L'agente alterna pianificazione e azione
- 1. Utile in ambienti dinamici o semidinamici
 - Non c'è troppo tempo per pianificare
- 2. Utile in ambienti non deterministici
 - 1. Pianificare vs agire
- 3. Necessaria per **ambienti ignoti** tipici dei problemi di **esplorazione**

Problemi di esplorazione

- I problemi di esplorazione sono casi estremi di problemi con contingenza in cui l'agente deve anche pianificare azioni esplorative
- Assunzioni per un problema di esplorazione:
 - Solo lo stato corrente è osservabile, l'ambiente è ignoto
 - Non si conosce l'effetto delle azioni e il loro costo
 - Gli stati futuri e le azioni che saranno possibili non sono conosciute a priori
 - Si devono compiere azioni esplorative come parte della risoluzione del problema
- Il labirinto come esempio tipico

Esempio: Teseo con mappa e senza

- Con mappa
 - applicabili tutti gli algoritmi di pianificazione visti
- Senza mappa
 - l'agente non può pianificare può solo esplorare nel modo più razionale possibile
 - Ricerca online



Assunzioni

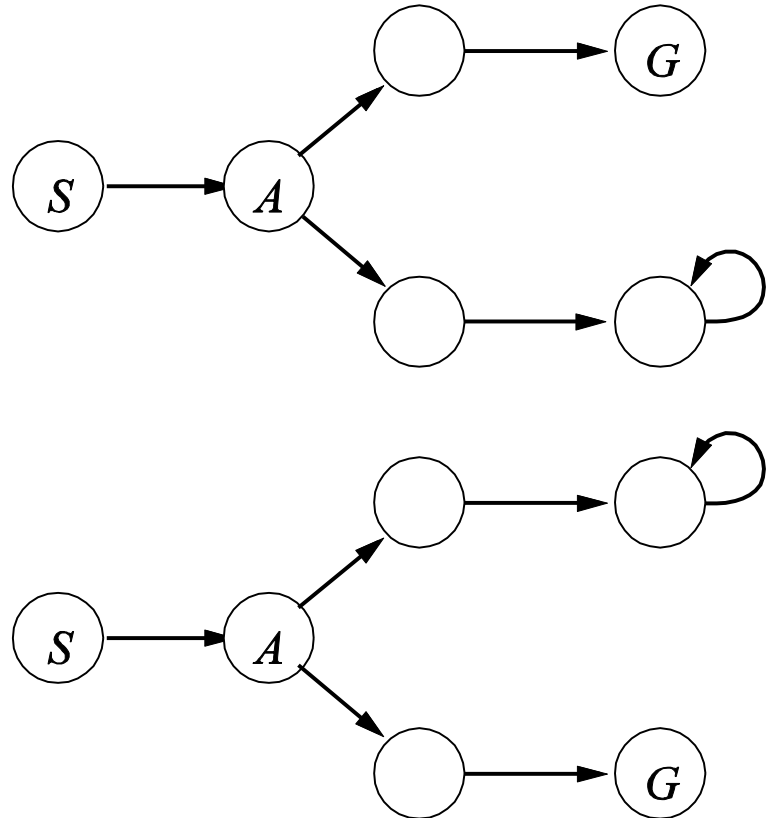
- Cosa conosce un agente *online* in s ...
 - Le azioni legali nello stato attuale s : Azioni (s)
 - Risultato(s, a), ma dopo aver eseguito a
 - Il costo della mossa $c(s, a, s')$, solo dopo aver eseguito a
 - Goal-test(s)
 - Può esserci: la stima della distanza: dal goal: $h(s)$

Costo soluzione

- Il costo del cammino è quello effettivamente percorso
- Il rapporto tra questo costo e quello ideale (conoscendo l'ambiente) è chiamato **rapporto di competitività**
- Tale rapporto può essere infinito
- Le prestazioni sono in funzione dello spazio degli stati

Assunzione ulteriore

- Ambienti **esplorabili in maniera sicura**
 - non esistono azioni irreversibili
 - lo stato obiettivo può sempre essere raggiunto
 - diversamente non si può garantire una soluzione



Ricerca in profondità *online*

- Gli agenti *online* ad ogni passo decidono l'azione da fare (non il piano) e la eseguono.
- Procedono solo “localmente”, ma anche otnando indietro e.g.:
- Ricerca in profondità *online*
 - Esplorazione sistematica delle alternative
 - $nonProvate[s]$ mosse ancora da esplorare in s
 - È necessario ricordarsi ciò che si è scoperto
 - $Risultato[s, a] = s'$
 - Il backtracking significa tornare sui propri passi
 - $backtrack[s]$ stati a cui si può tornare

Esempio

- Sceglie il primo tra $(1,1)$ e $(2,2)$
- In $(1, 1)$ ha solo l'azione per tornare indietro (POP backtrack[s])
- ...
- Nella peggiore delle ipotesi esplora ogni casella due volte

	1	2	3	4
1				
2				T →
3				
4				

Algoritmo in profondità *online*

```
function Agente-Online-DFS(s) returns un'azione
    static: Risultato, nonProvate, backtrack,
            s- (stato precedente), a- (ultima azione)
    if Goal-Test(s) then return stop
    if s è un nuovo stato then nonProvate[s]  $\leftarrow$  Azioni(s)
    if s- non è null then risultato[s-, a-]  $\leftarrow$  s; backtrack[s]  $\leftarrow$  s-;
    if nonProvate[s] vuoto then
        if backtrack[s] vuoto then return stop
        else a  $\leftarrow$  azione per tornare in POP(backtrack[s])
    else a  $\leftarrow$  POP(nonProvate[s])
    s-  $\leftarrow$  s; return a
```

Ricerca euristica *online*

- Nella ricerca online si conosce il valore della funzione euristica una volta esplorato lo stato.
- Un algoritmo di tipo Best First non funzionerebbe.
- Serve un metodo locale
- *Hill-climbing* con *random-restart* non praticabile
- Come sfuggire a minimi locali?

Due soluzioni

1. Random-walk

- si fanno mosse casuali (privilegiando le nuove)
- in principio completa ma costosa!

2. Ricerca locale con A^* (LRTA*):

- Learning Real Time A^* , A^* con memoria (apprendimento) in tempo reale
- esplorando si aggiustano i valori dell'euristica per renderli più realistici
- In questo modo riesce a superare i minimi locali

Idea dell'algoritmo LRTA*

- $H(s)$: migliore stima trovata fin qui
- Si valutano i successori:

$$\text{Costo-LRTA}^*(s, a, s', H) =$$

$h(s)$ se s' indefinito (non esplorato)

$H(s') + \text{costo}(s, a, s')$ altrimenti

- Ci si sposta sul successore di Costo-LRTA* minore
- Si aggiorna la H dello stato da cui si proviene

LRTA*

function Agente-LRTA*(s) **returns** un'azione

static: risultato, H, s-, a-

if Goal-Test(s) **then return** stop

if s nuovo (non in H) **then** $H[s] \leftarrow h[s]$

1. **if** s- non è null //si aggiusta il costo H del predecessore

$\text{risultato}[s-, a-] \leftarrow s$

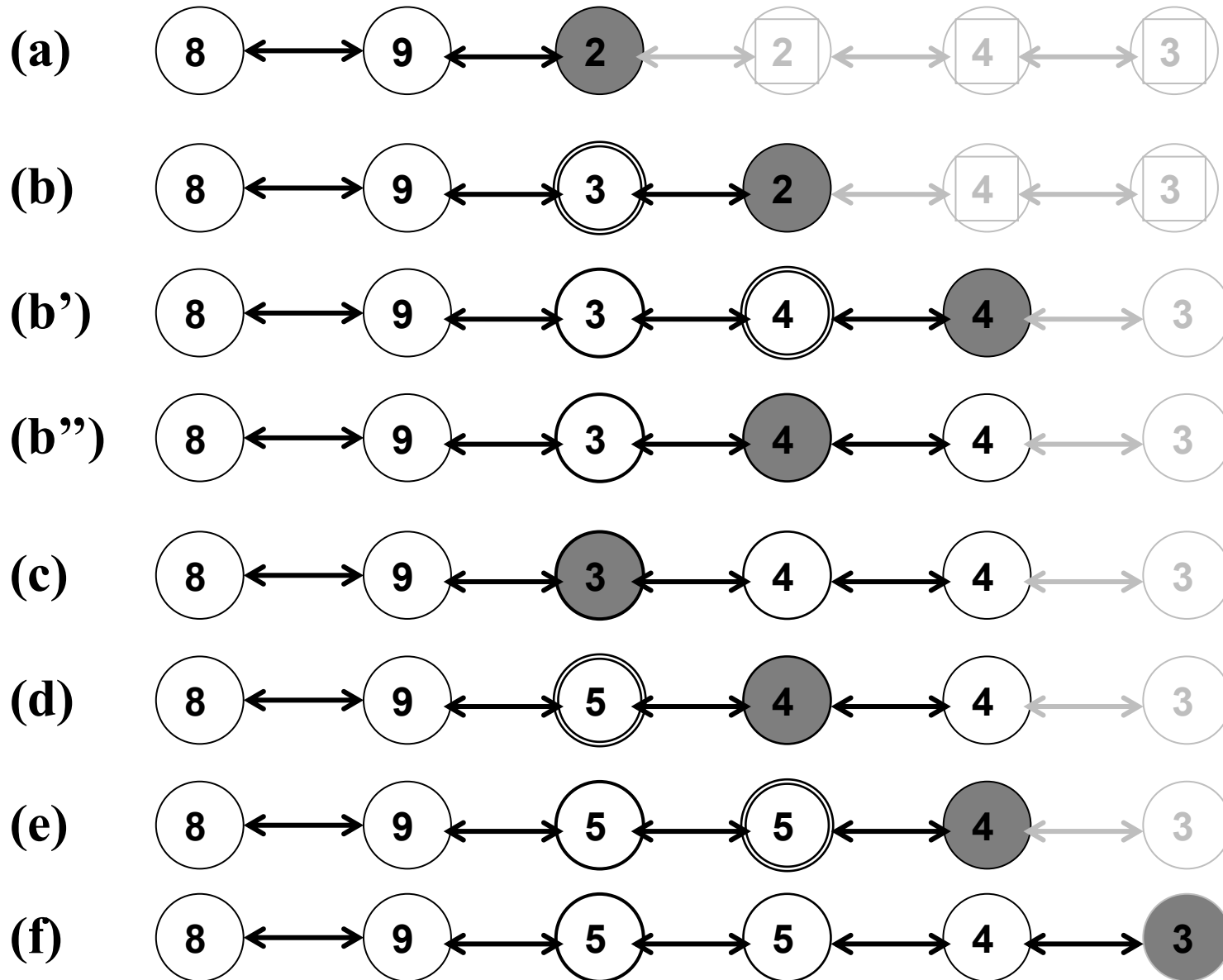
$H[s-] \leftarrow \min \text{Costo-LRTA}^*(s-, b, \text{risultato}[s-, b], H)$

2. $a \leftarrow \text{un'azione } b \text{ tale che minimizza}$ $b \in \text{Azioni}(s-)$

$\text{Costo-LRTA}^*(s, b, \text{risultato}[s, b], H)$

$s- \leftarrow s$; **return** a

LRTA* supera i minimi locali (rev)



Esempio di LRTA*

	1	2	3	4
1	(H=4)			
2	(H=3)	(H=2)	(H=3)	T (H=0) →
3		(H=3)	(H=4)	(H=1)
4			(H=3)	(H=2)

Considerazioni su LRTA*

- LRTA* cerca di simulare A* con un metodo locale: tiene conto del costo delle mosse come può aggiornando al volo la H
- Completo in spazi esplorabili in maniera sicura
- Nel caso pessimo visita tutti gli stati due volte ma è mediamente più efficiente della profondità *online*
- Non ottimale, a meno di usare una euristica perfetta (non basta una $f=g+h$ con h ammissibile)
- Verso l'***apprendimento per rinforzo!***